Machine Learning – An Overview

Fundamentals, Methods, and Applications in Gretl

Artur Tarassow

Applied University of Brandenburg
artur.tarassow@th-brandenburg.de

PhD Seminar in Ancona (Erasmus)

Outline

1. Introduction: What is Machine Learning?
2. Supervised Learning: Foundations
3. OLS, Ridge and Lasso
4. K-Nearest Neighbors
5. Supervised Learning: Tree-Based and Kernel Methods
6. Support Vector Machines
7. Cross-Validation and Model Evaluation
   - CV for Classification Problems
   - CV for Time-Series Problems
   - Evaluation Metrics for Regression
   - Evaluation Metrics for Classification
8. Unsupervised Learning: Clustering

Learning Objectives

- Understand fundamental ML concepts: supervised vs. unsupervised learning
- Apply key supervised learning methods: OLS, Ridge, Lasso, KNN, Trees, Random Forests, SVM
- Master cross-validation and model evaluation techniques
- Implement practical examples in Gretl
- Interpret feature importance and make informed method selection

ML Fundamentals Overview

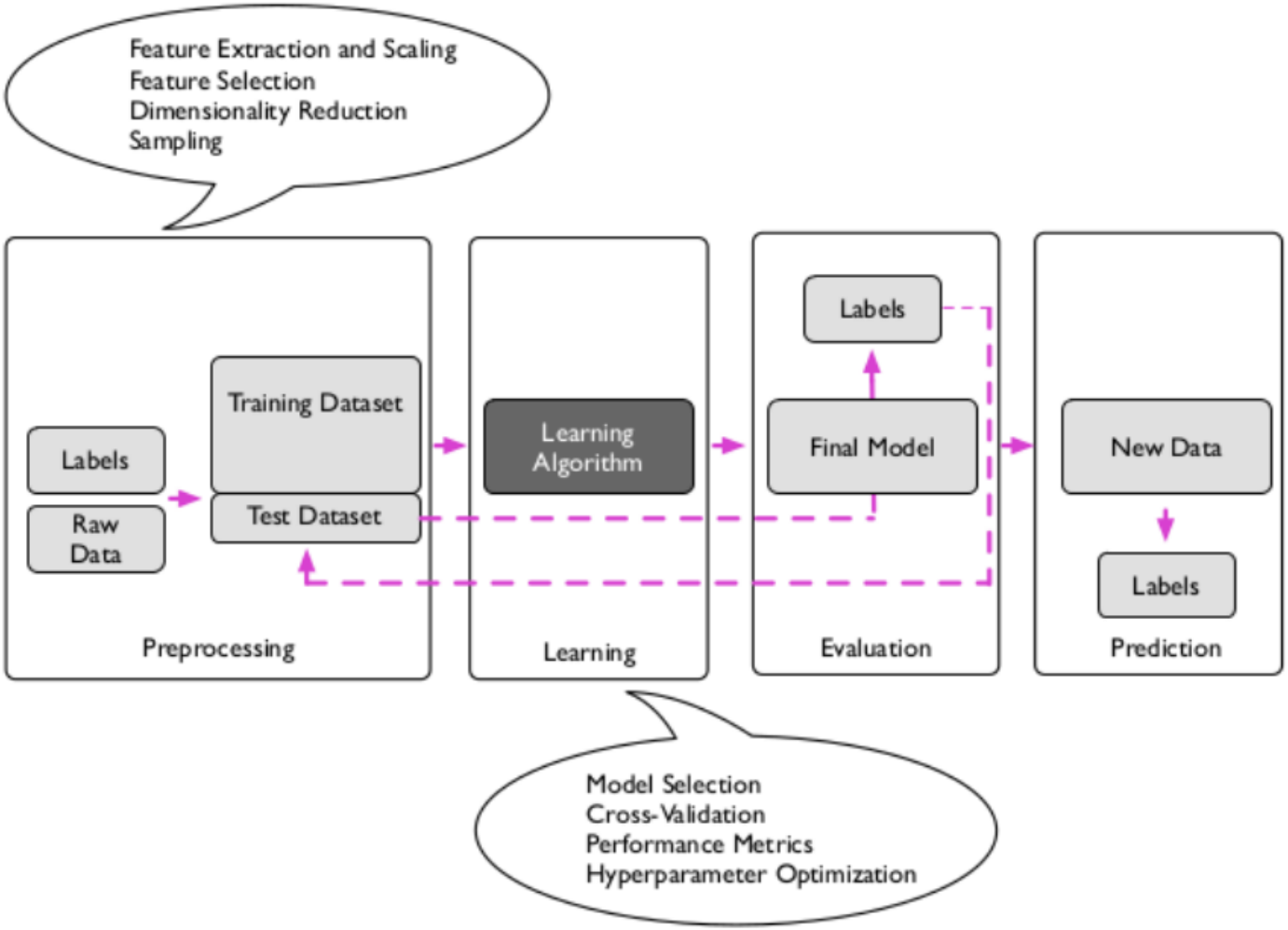


Figure 1: Roadmap for building ML systems. Source: (Raschka and Mirjalili, 2019, p. 11).

References

- Bishop (2016) — Pattern Recognition and Machine Learning (PRML)
- James et al. (2023) — Introduction to Statistical Learning (ISL)
- Hastie et al. (2009) — Elements of Statistical Learning (ESL)
- Raschka and Mirjalili (2019) — Python Machine Learning
- Yu and Barter (2024) — Veridical Data Science: The Practice of Responsible Data Analysis and Decision Making
- Athey & Wager (2019) — Machine Learning Methods for Economists

Technische Hochschule
Brandenburg
University of
Applied Sciences
Fachbereich
Wirtschaft

Motivation and Context

Why Machine Learning for Economics?

What is Machine Learning?

Definition

Machine learning refers to a set of computational methods that **learn from data** without being explicitly programmed. The system improves its performance on a given task through experience.

Three Learning Paradigms:

1. **Supervised Learning**: Learn from labeled data with known output signals (labels)
   - Regression (continuous output), Classification (discrete output)
2. **Unsupervised Learning**: Find patterns in unlabeled data
   - Clustering, Dimensionality reduction
3. **Reinforcement Learning**: Learn through interaction and rewards
   - Not covered in this seminar

ML vs. Statistics vs. Econometrics

Dimension	**Statistics**	**ML**	**Econometrics**
Primary Goal	Inference	Prediction	Causal Inference
Focus	Population parameters	Generalization	Structural relationships
Uncertainty	p-values, CI	Validation error	Significance tests
Assumptions	Rigid	Flexible	Economic theory
Interpretability	Essential	Optional	Critical
Data Dimension	Low-medium	High	Low-medium

Key insight: *Good prediction requires controlling overfitting and model complexity, not just fitting the observed data well.*

Why ML for Economics?

- **High-dimensional problems**: Modern datasets have many features
  - Example: Predicting firm defaults with hundreds of financial indicators
- **Nonlinear relationships**: Economic phenomena often lack linearity
  - Example: Household consumption behavior with interaction effects
- **Model selection**: Regularization helps choose among competing specifications
  - Replaces ad-hoc variable selection with automatic methods
- **Prediction accuracy**: Sometimes prediction matters more than understanding
  - Example: Forecasting macroeconomic variables for policy decisions
- **Causal ML**: Combines econometric theory with ML flexibility
  - Example: Heterogeneous treatment effects, policy evaluation

Technische Hochschule
Brandenburg
University of
Applied Sciences
Fachbereich
Wirtschaft

Supervised Learning

Core Concepts and Methods

Regression vs. Classification

Supervised Learning Problem:

$$y = f(\mathbf{X}) + \varepsilon$$

where $\mathbf{X} = (X_1, \ldots, X_p)$ are features/predictors and y is the target/response.

Regression

- $y \in \mathbb{R}$ (continuous)
- Goal: Predict a real number
- Example: House prices, GDP growth
- Error: MSE, MAE, RMSE

Classification

- $y \in \{0, 1, \ldots, K-1\}$ (discrete)
- Goal: Predict a category
- Example: Default/no default, recession/no recession
- Error: Misclassification rate, AUC

Training, Validation, and Test Data

Standard procedure to estimate generalization error:

1. **Training Set** ($\sim 70\%$ of data)
   - Fit all candidate models here
   - Choose model complexity (e.g., λ) via validation error
2. **Validation Set** ($\sim 15\%$ of data)
   - Tune hyperparameters (e.g., regularization parameter)
   - Select among candidate models
3. **Test Set** ($\sim 15\%$ of data)
   - **Never touch during training**
   - Report final performance here
   - Represents future unseen data

Why? Training error is overly optimistic due to overfitting. Test error is the real performance metric.

Bias-Variance Tradeoff: Visual Example

Variance: Consistency of the model prediction for a data point if we would retrain the model multiple times on different subsets of the training data. ⇒ Sensitivity to randomness in the data.

Bias: Distance of the average prediction of the model from the true value we want to predict. ⇒ Systematic error not due to randomness but due to model assumptions.

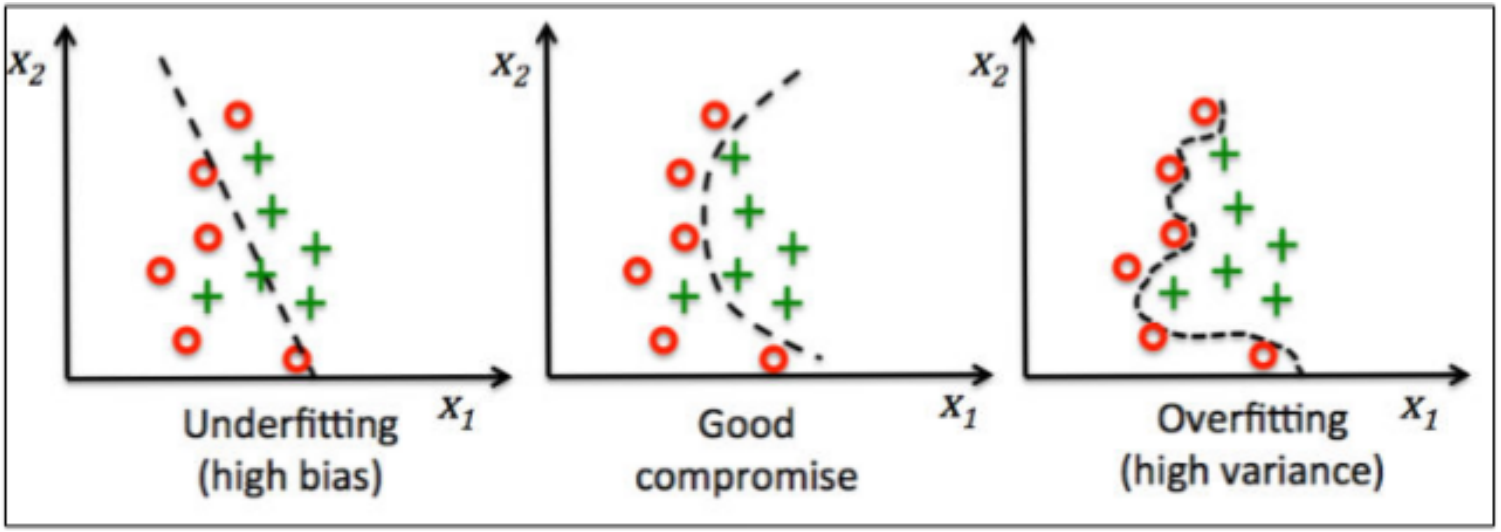


Figure 2: Example of underfitting (high bias), good fit (balanced), and overfitting (high variance). Source: (Raschka and Mirjalili, 2019, p. 66).

Bias-Variance Tradeoff

Test error (expected loss) decomposition: (Bishop, 2016, pp. 148)

$$\text{Test Error} = \underbrace{\text{Bias}^2}_{\text{underfitting}} + \underbrace{\text{Variance}}_{\text{overfitting}} + \underbrace{\text{Irreducible Error}}_{\text{noise}}$$

- **High Bias**: Model too simple, cannot capture true relationship
  - Linear regression on nonlinear data
- **High Variance**: Model too flexible, fits noise in training data
  - Very deep decision tree overfitting training data
- **Objective**: Minimize expected loss
- **Problem**: Tradeoff between bias and variance — reducing one often increases the other
- **Goal**: Balance — find "sweet spot" complexity

Technische Hochschule
Brandenburg
University of
Applied Sciences
Fachbereich
Wirtschaft

OLS, Ridge, and Lasso Regression

OLS

$$\min_{\beta} \frac{1}{n} \sum_{i=1}^{n} (y_i - \mathbf{x}_i^\top \beta)^2 \tag{1}$$

- Baseline linear model
- No regularization — fits training data as closely as possible
- High bias if omitted variables are important; high variance with many features
- Solution: $\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$

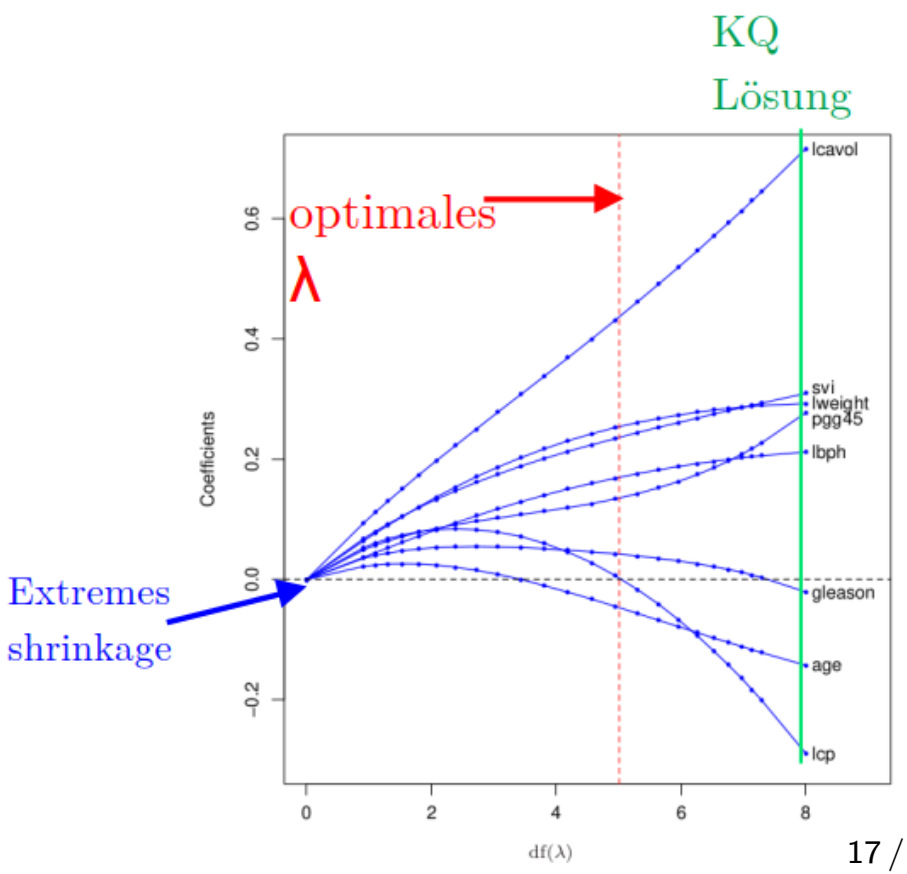
When does OLS fail?

- Many features ($p > n$): $\mathbf{X}^\top \mathbf{X}$ becomes singular
- Multicollinearity: Coefficients become large and unstable

Regularization: Ridge Regression (L2 Penalty)

$$\min_{\beta} \frac{1}{n} \sum_{i=1}^{n} (y_i - \mathbf{x}_i^\top \beta)^2 + \underbrace{\lambda \sum_{j=1}^{p} \beta_j^2}_{\text{L2 penalty}} \tag{2}$$

- **Penalty term**: shrinks coefficients in contrast to discrete variable selection
- **Tuning parameter** $\lambda \geq 0$ controls shrinkage strength
- $\lambda = 0 \Rightarrow$ OLS; $\lambda \to \infty \Rightarrow \beta_j \to 0$
- Reduces variance at the cost of small bias
- **No feature selection**: coefficients are typically not set to zero



Regularization: Lasso Regression (L1 Penalty)

$$\min_{\beta} \frac{1}{n} \sum_{i=1}^{n} (y_i - \mathbf{x}_i^\top \beta)^2 + \underbrace{\lambda \sum_{j=1}^{p} |\beta_j|}_{\text{L1 penalty}} \tag{3}$$

- **Penalty term**: L1 norm, not squared
- **Key difference from Ridge**: Induces **sparsity**
- Some coefficients shrunk **exactly to zero** — automatic feature selection
- Lasso can be inconsistent in feature selection (Zou, 2006) – adaptive lasso can fix this
- Useful for high-dimensional problems ($p \gg n$)

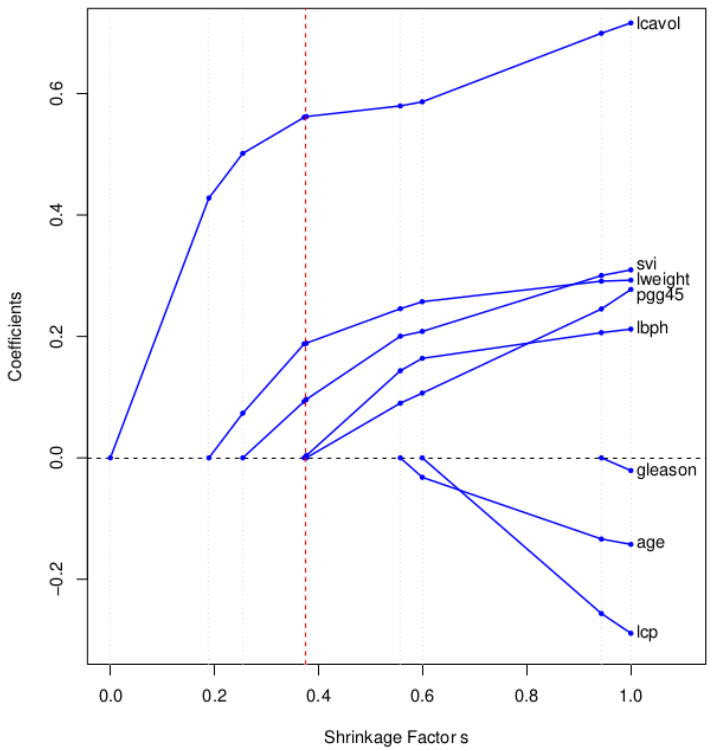


FIGURE 3.10. *Profiles of lasso coefficients, as the tuning parameter t is varied. Coefficients are plotted versus* $s = t/\sum_1^p |\hat{\beta}_j|$. *A vertical line is drawn at* $s = 0.36$, *the value chosen by cross-validation. Compare Figure 3.8 on page 65; the lasso*

Gretl Packages for Regularized Regression

Gretl packages

- `regls`: Add-on for Ridge and Lasso regression (Allin Cottrell).
  Doc: https://sourceforge.net/projects/gretl/files/addons/doc/regls.pdf
- `graphlasso`: Graphical Lasso (Sven Schreiber).
  Doc: https://gretl.sourceforge.net/current_fnfiles/graphlasso.gfn
- `adalasso`: Adaptive Lasso (Sven Schreiber).
  Doc: https://gretl.sourceforge.net/current_fnfiles/adalasso.gfn
- `fsboost`: Forward-stagewise boosted regression estimates (Artur Tarassow).
  Doc: https://gretl.sourceforge.net/current_fnfiles/unzipped/fsboost.pdf

Code Example 1: OLS, Ridge, and Lasso in Gretl

See code examples:

- `ridge_lasso_coefficient_path.inp`: Coefficient paths for Ridge and Lasso using the `regls` package
- `crossval_ridge_lasso.gretl`: Cross-validation to select λ for Ridge and Lasso

Technische Hochschule
Brandenburg
University of
Applied Sciences
Fachbereich
Wirtschaft

K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN): Lazy Learning

- Popular non-parametric approach in machine learning.
- *Lazy learner*: it does not learn an explicit function but memorizes the training dataset (*instance-based learner*).
- Basic idea: similar data points (features) should have similar outcomes (target values).
- Identifies the K points in the training data that are closest to a new point x_q (query).
- **Advantage** of the memory-based approach: the model immediately adapts to new data as soon as it becomes available.
- **Disadvantage**: Computational complexity increases linearly with n^{train}.

The Algorithm – Illustrated for $K = 1$ closest neighbor

Training dataset $\mathcal{D}$:
$\{\mathbf{x}_i, y_i\} \in \mathcal{D} \quad (|\mathcal{D}| = n)$
New point: $\{\mathbf{x}_q, ???\}$ (query point)
Prediction: $f(\mathbf{x}_q)$

Algorithm

closest_point = none
closest_distance = ∞

1. for $i = 1, \ldots, n$
    2. current_distance = $d(\mathbf{x}_i, \mathbf{x}_q)$
    3. if current_distance < closest_distance
        4. closest_distance = current_distance
        5. closest_point = $\mathbf{x}_i$
6. return $f(closest_points)$ in the space $\mathcal{D}_k$.

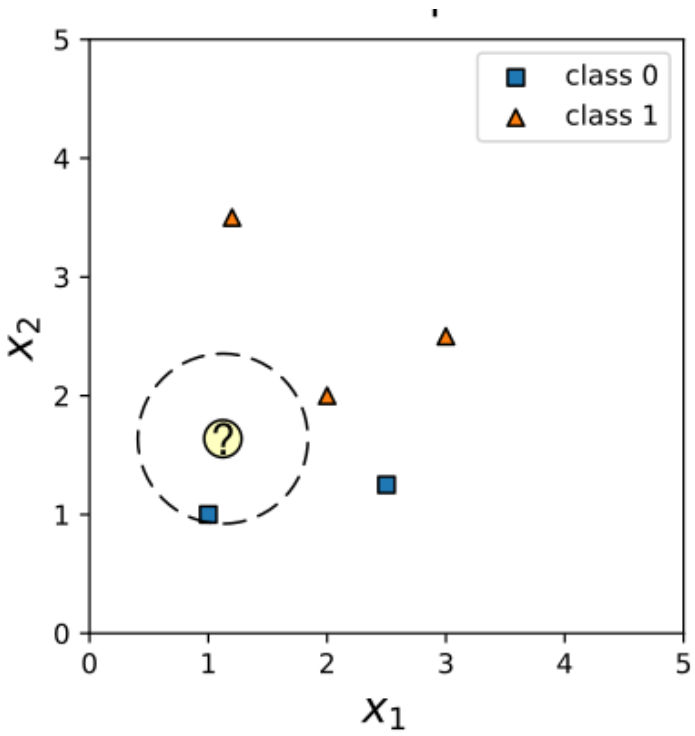


Figure 3: Search for the nearest data point in the training dataset ($n = 5$) which looks most "similar" using some distance function $d()$.

Prediction for query point $\mathbf{x}_q$

- For a new observation $\mathbf{x}_0$, find the k closest observations in training data
  - Distance metric: Usually Euclidean: $d(\mathbf{x}_q, \mathbf{x}_i) = \sqrt{\sum_j (x_{qj} - x_{ij})^2}$
- **For classification**: Majority vote among k neighbors
  - $\hat{y}_0 = \text{mode}(y_1, \ldots, y_k)$
- **For regression**: Average the k neighbors
  - $\hat{y}_0 = \frac{1}{k} \sum_{i=1}^{k} y_i$

Hyperparameter: k (number of neighbors)

- Small k (e.g., 1): Flexible, high variance, overfitting risk
- Large k (e.g., n): Inflexible, high bias, underfitting risk
- Typically selected via cross-validation

KNN: Advantages and Disadvantages

Advantages:

- Simple and intuitive
- No assumptions about data distribution
- Works for both regression and classification
- Captures local patterns well

Disadvantages:

- **Curse of dimensionality**: Distance becomes less meaningful with many features
- Computational cost: Must compute distance to all training points for each prediction
- Sensitive to feature scaling — requires standardization
- Poor interpretability — why did we classify this way?

Class separability & Hyperparameter Tuning Using CV

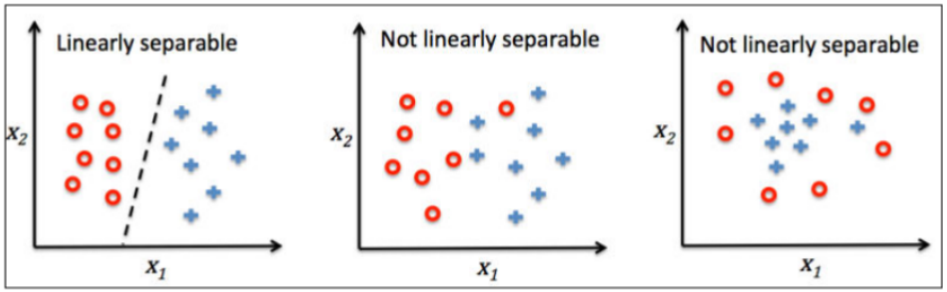


Figure 4: Linear vs. non-linear separability. Source: (Raschka and Mirjalili, 2019, p. 23).

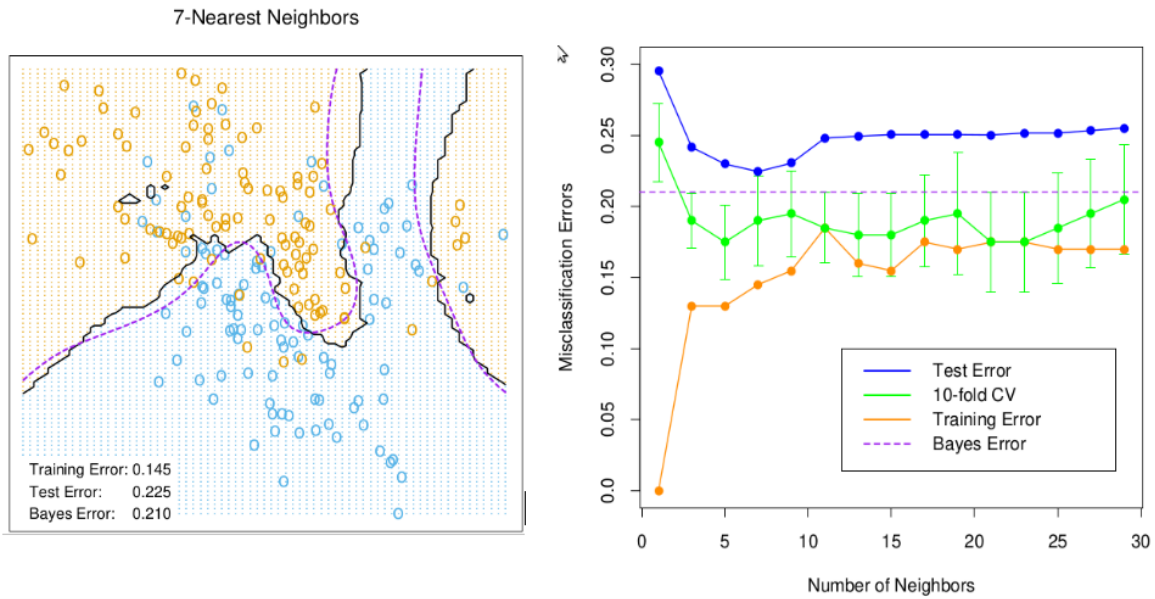


FIGURE 13.4. *k-nearest-neighbors on the two-class mixture data. The upper panel shows the misclassification errors as a function of neighborhood size. Standard error bars are included for 10-fold cross validation. The lower panel shows the decision boundary for 7-nearest-neighbors, which appears to be optimal for minimizing test error. The broken purple curve in the background is the Bayes*

KNN Using Time-Series Data

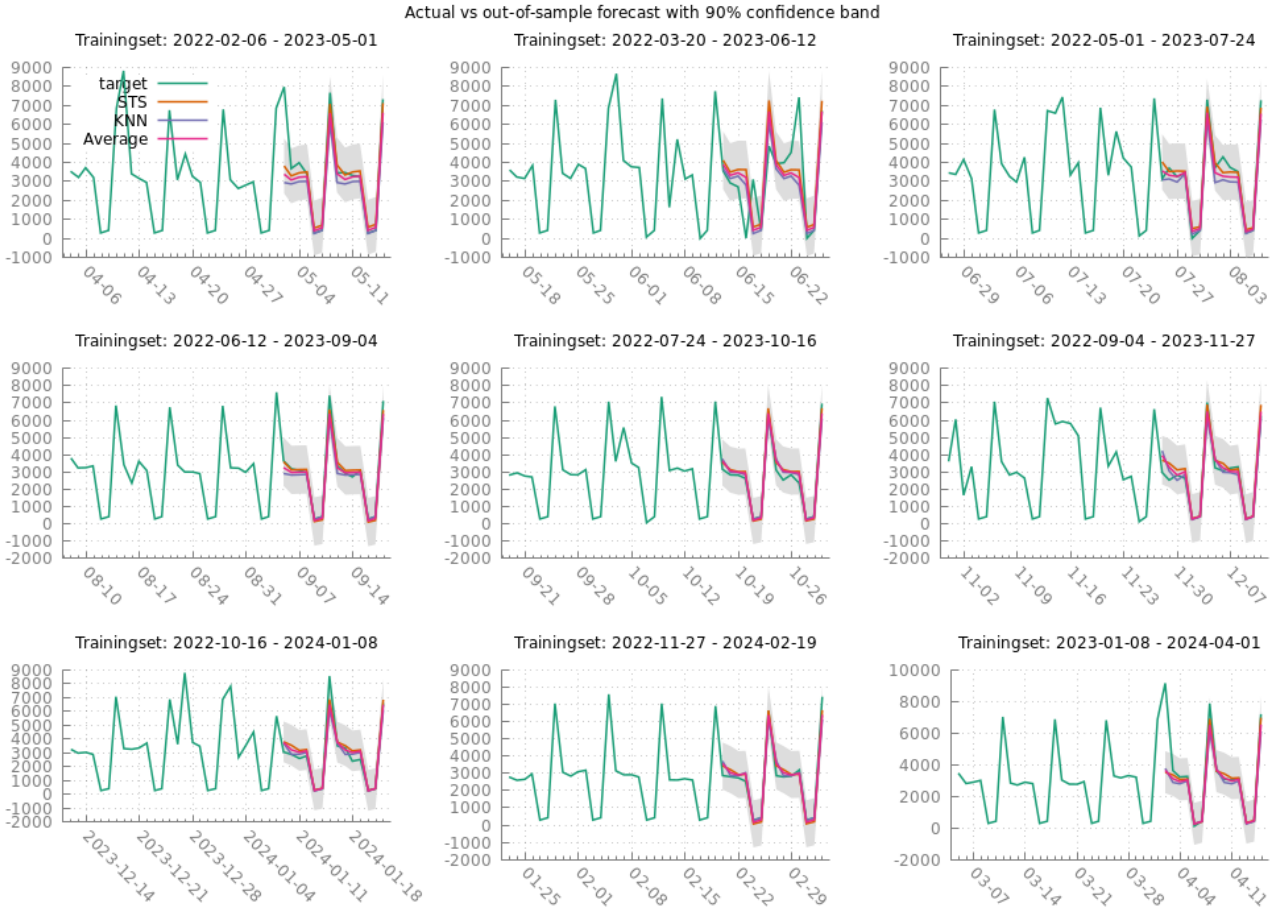


Figure 6: *Out-of-sample* ($h = 14$ days) time-series forecasting using KNN, in comparison to a structural time-series model. Daily data with day features (e.g., day of week, holidays).

Gretl Package for KNN

Package: `knn.gfn` (Author: Artur Tarassow, v0.1, July 2024)
Main Features:

- **Dual functionality:** Regression and binary/multi-class classification
- **Core functions:**
  - `knn_fit()`: Fit KNN model to training data
  - `knn_predict()`: Make predictions on test data
  - `knn_scores()`: Calculate accuracy metrics
  - `knn_summary()`: Print model summary
- **Distance metrics:** Various
- **Cross-validation:** K-fold, leave-one-out, recursive/rolling windows

Installation: `pkg install knn`

Code Example 2: K-Nearest Neighbors in Gretl

File: `../code/02_knn_classification.inp`

```
#
    ==================================================================
# Example 2: K-Nearest Neighbors Classification
# Package: knn.gfn (Artur Tarassow)
# Author: PhD Seminar, Ancona 2026
#
    ==================================================================



# Install and include package
pkg install knn --quiet
include knn.gfn

# Load data
open ../data/TODO_dataset.gdt  # TODO:
    Classification dataset
```

Technische Hochschule
Brandenburg
University of
Applied Sciences
Fachbereich
Wirtschaft

Tree-Based Methods

Decision Trees, Random Forests, and Beyond

Motivation

- Tree-based methods as the *workhorse* of supervised machine learning
- Used for both **regression** and **classification**
- Flexible non-parametric approach to data analysis and prediction
- Recursive division of the feature space into regions (*stratification*/*segmentation*)
- Rule-based approach — **rules are learned by the algorithm**, not hard-coded
- Each region assigned a single prediction value
- Interpretable: Can be visualized and explained
- Lacking prediction accuracy for a *single* tree but forms the basis for powerful ensemble methods (Random Forests, Gradient Boosting)

Decision Tree

- The tree learns a sequence of (binary) questions based on features to identify the target variable classes
- Each question is represented by a node in the tree
- The leaves of the tree represent the classes
- Categorical variables and numerical variables can be used for splitting

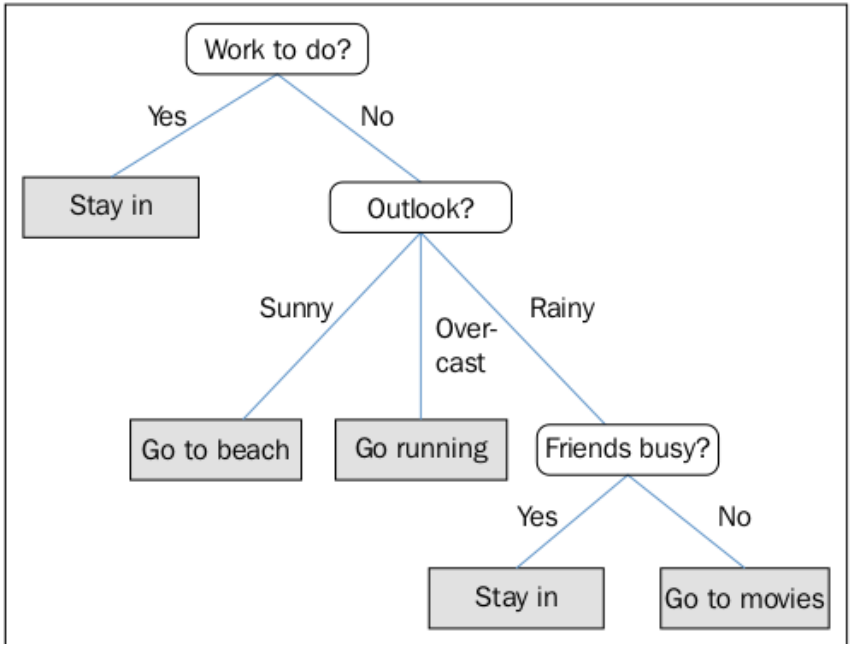


Figure 7: Decision tree for day planning. Source: Raschka and Mirjalili (2019)

Basic Concepts of Decision Trees

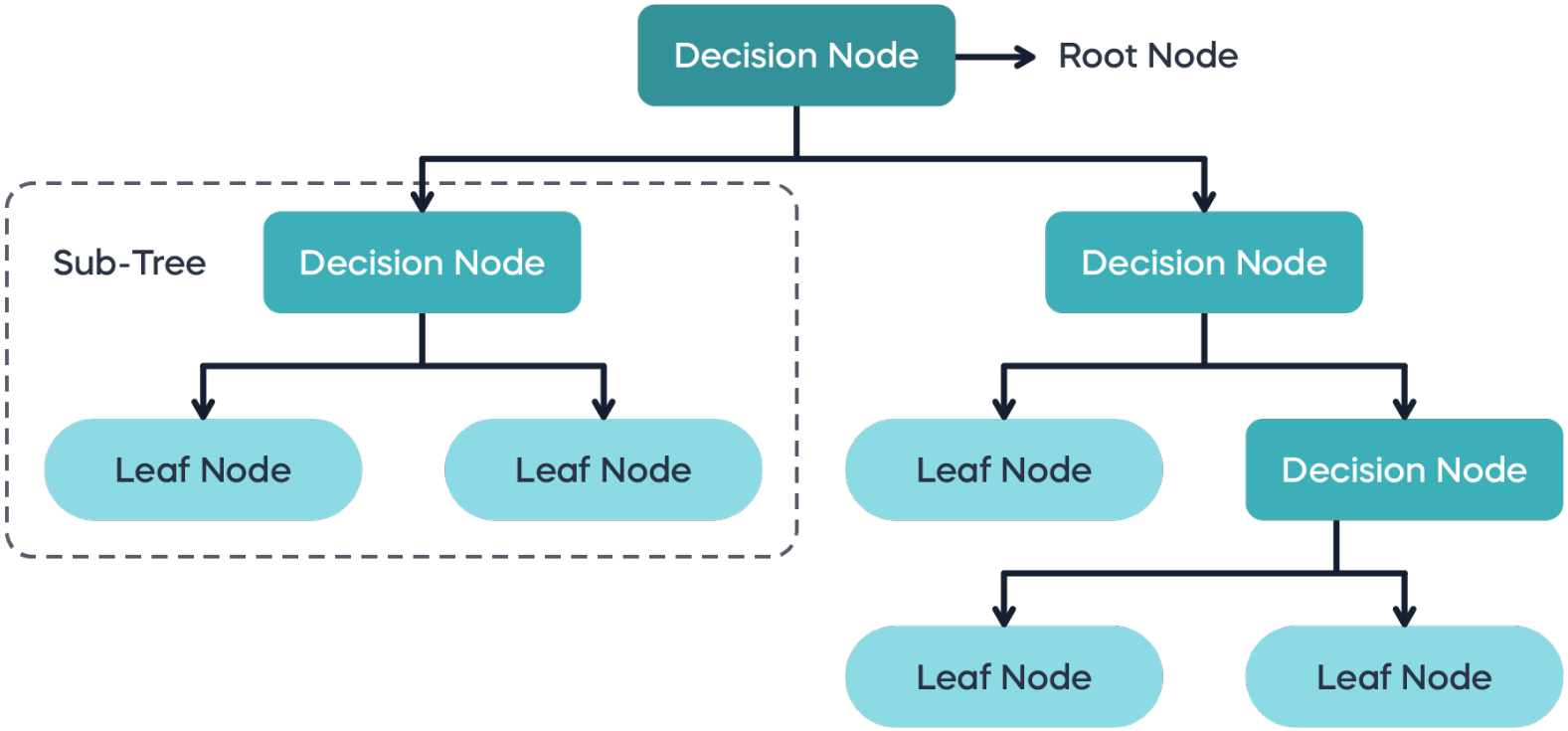


Figure 8: Connections between decision nodes are called **branches**.
Source: https://365datascience.com/tutorials/machine-learning-tutorials/decision-trees/

Example: Modeling Baseball Player Salaries I

The CART algorithm is a popular method for building decision trees (Breiman et al., 1984).

How it works:

1. Start with all data at root node
2. Choose split on feature that best reduces impurity (Gini, entropy)
3. Recursively split child nodes until stopping criterion
4. Assign prediction based on majority (classification) or mean (regression)

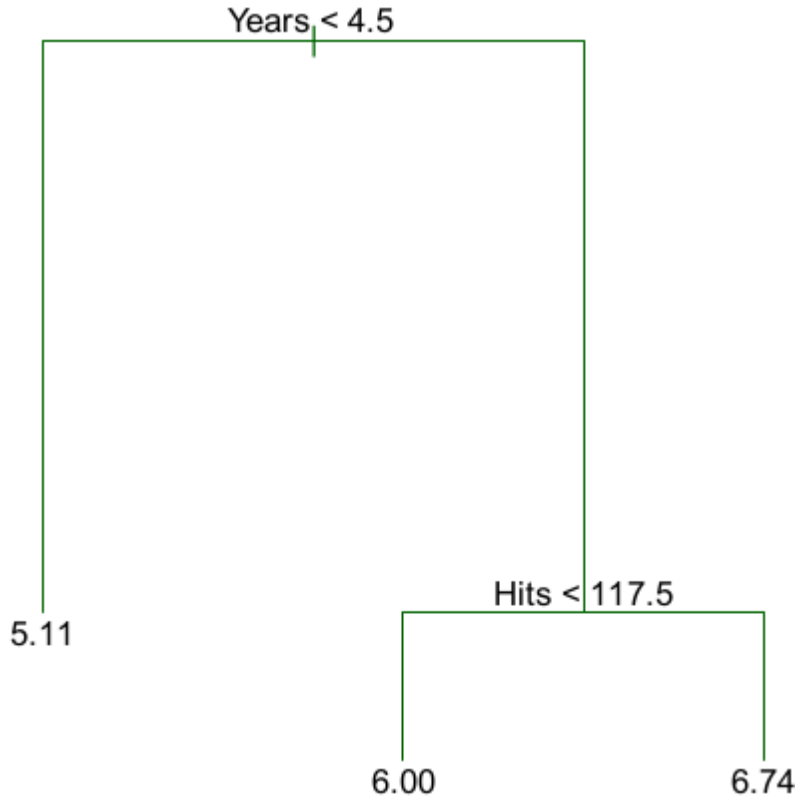


Figure 9: Binary tree with 2 internal nodes and 3 leaves. The value of each leaf corresponds to the average (logarithmic) salary of players in this partition.

Example: Modeling Baseball Player Salaries II

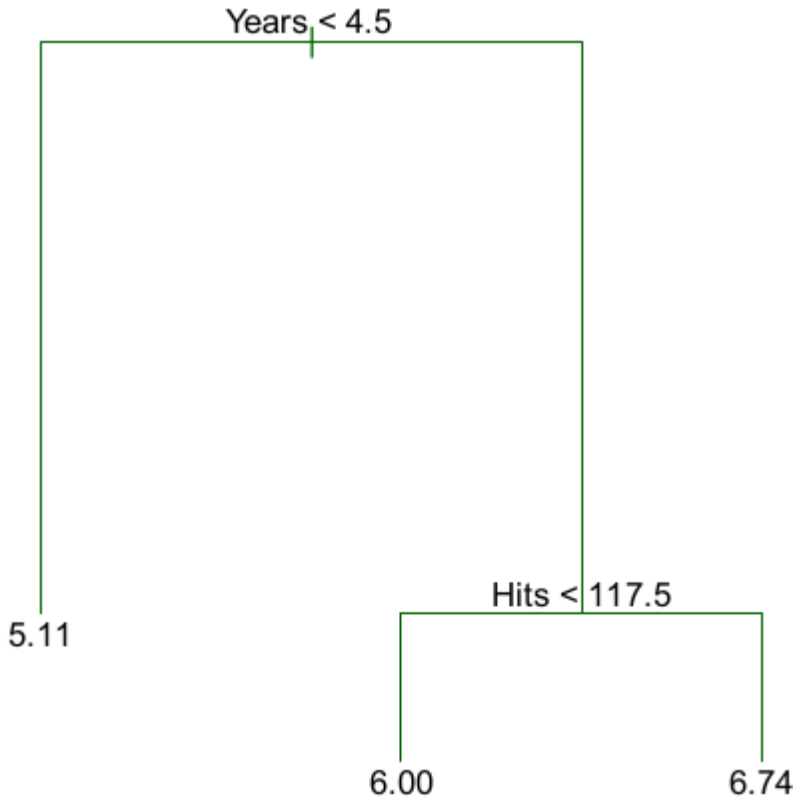


Figure 10: Players with less than 4.5 years of playing experience earn an average of $e^{5.11} = 165,000$ USD.

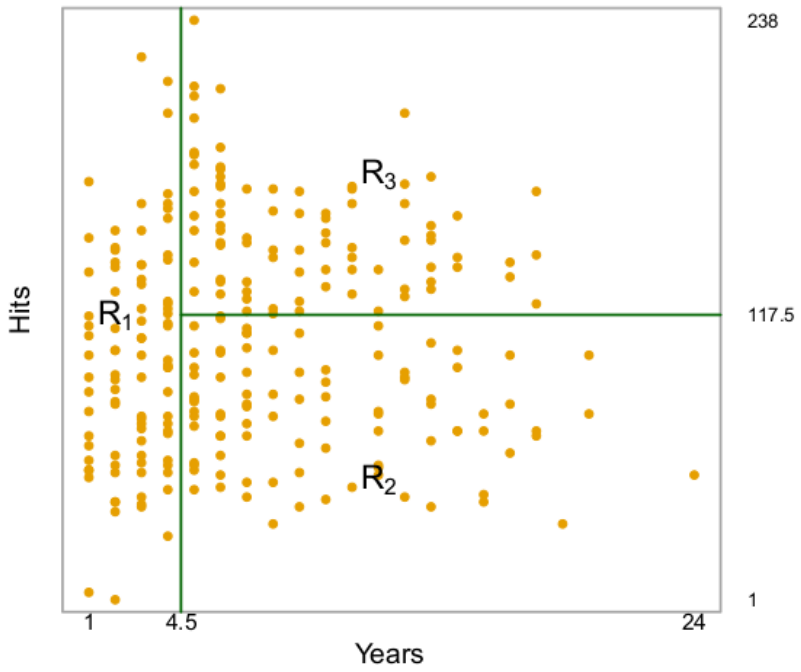


Figure 11: The three regions of the input space defined by the tree in Fig. 10. This is the optimally *pruned* tree with 3 terminal nodes.

Decision Trees: Splitting Criteria

How to choose the best split?[1]

For Classification — minimize impurity:

- **Gini Impurity**: $G = 1 - \sum_{k=1}^{K} p_k^2$ (proportion of class k in node)
  - Ranges 0 (pure, all one class) to $1 - 1/K$ (maximally mixed)
- **Entropy**: $H = -\sum_{k=1}^{K} p_k \log(p_k)$ (information theoretic)
  - Higher entropy = more uncertainty

For Regression — minimize within-node variance (look for similar y values):

- For a split into left (L) and right (R) child nodes:

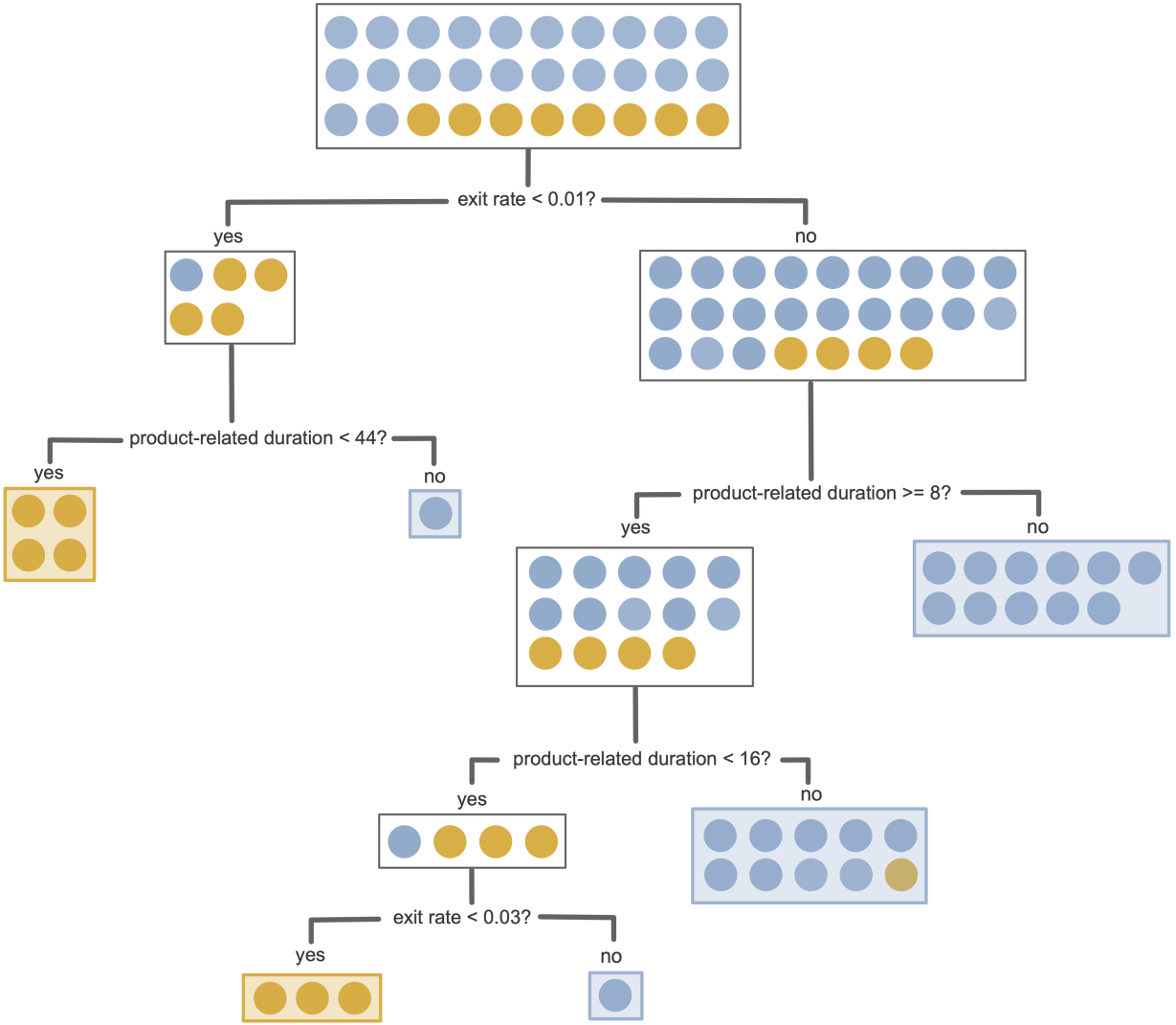
$$\text{Var}_{\text{split}} = \frac{n_L}{n} \times \sigma_L^2 + \frac{n_R}{n} \times \sigma_R^2$$

where n_L, n_R are the number of observations in the left/right child nodes, $n = n_L + n_R$ is the parent node size, and σ_L^2, σ_R^2 are squared deviations from the mean in each child node, $\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(y_i - \bar{y})^2$.

[1] For details, see Yu and Barter (2024). URL: https://vdsbook.com/12-rf

Binary Classification Tree for a Purchase Decision

A diagram depicting a decision tree for predicting purchase intent (a binary response). The tree has been trained on 30 training sessions, 8 of which ended with a purchase and are colored orange, and 22 of which did not and are colored blue.
Source



Decision Trees: Overfitting and Pruning

Problem: Trees can grow very deep, fitting noise in training data

Unpruned Tree

- Zero training error
- High test error (overfitting)
- Captures noise

Pruned Tree

- Small training error
- Lower test error (better generalization)
- Simpler, more interpretable

Key Hyperparameters (CART Algorithm):

Hyperparameter	Description	R (rpart)	Python (sklearn)
Maximum depth	Max number of sequential splits	`maxdepth` (default: 30)	`max_depth` (no default)
Minimum node size	Min observations to split a node	`minsplit` (default: 20)	`min_samples_split` (default: 2)

Comparison of Model Types

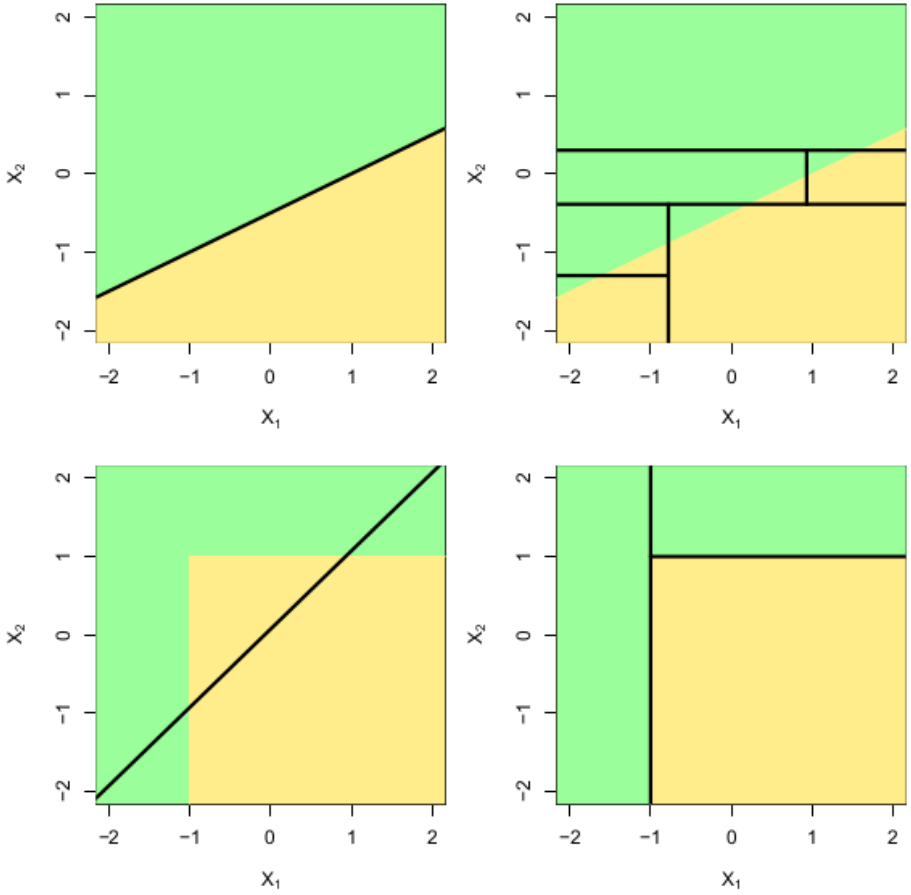


FIGURE 8.7. Top Row: *A two-dimensional classification example in which the true decision boundary is linear, and is indicated by the shaded regions. A classical approach that assumes a linear boundary (left) will outperform a decision tree that performs splits parallel to the axes (right).* Bottom Row: *Here the true decision boundary is non-linear. Here a linear model is unable to capture the true decision boundary (left), whereas a decision tree is successful (right).*

Code Example 3: Decision Trees in Gretl

File: `../code/03_decision_tree.inp`

```
# ==================================================================
# Example 3: Decision Tree for Classification/Regression
# Note: Gretl has limited built-in tree support
# Author: PhD Seminar, Ancona 2026
# ==================================================================

# TODO: Determine exact Gretl tree-building function
# Options:
#   1. Use R integration via rpart package
#   2. Use external tree package if available
#   3. Demonstrate concept with logit/probit tree splits
```

Random Forests: Bootstrap Aggregation

Problem with single tree: High variance, overfitting

Solution: Grow many **decorrelated** trees and average (aggregate) their predictions (Breiman, 1996; Breiman, 2001)

Algorithm:

1. For $b = 1, \ldots, B$ (number of trees):
   - Draw bootstrap sample of size n with replacement
   - Grow full tree on bootstrap sample (no pruning)
   - At each split, randomly select m features from p total
     - Typical: $m = \sqrt{p}$ for classification, $m = p/3$ for regression
2. Average predictions across all B trees:

   Classification (binary response):
   - *Probability prediction*: Average of positive class probability predictions from individual trees
   - *Binary class prediction*: Majority vote of individual trees' binary predictions (or thresholded class probability)
3. **Regression (continuous response)**: Mean prediction across all individual trees

Random Forests: Illustration

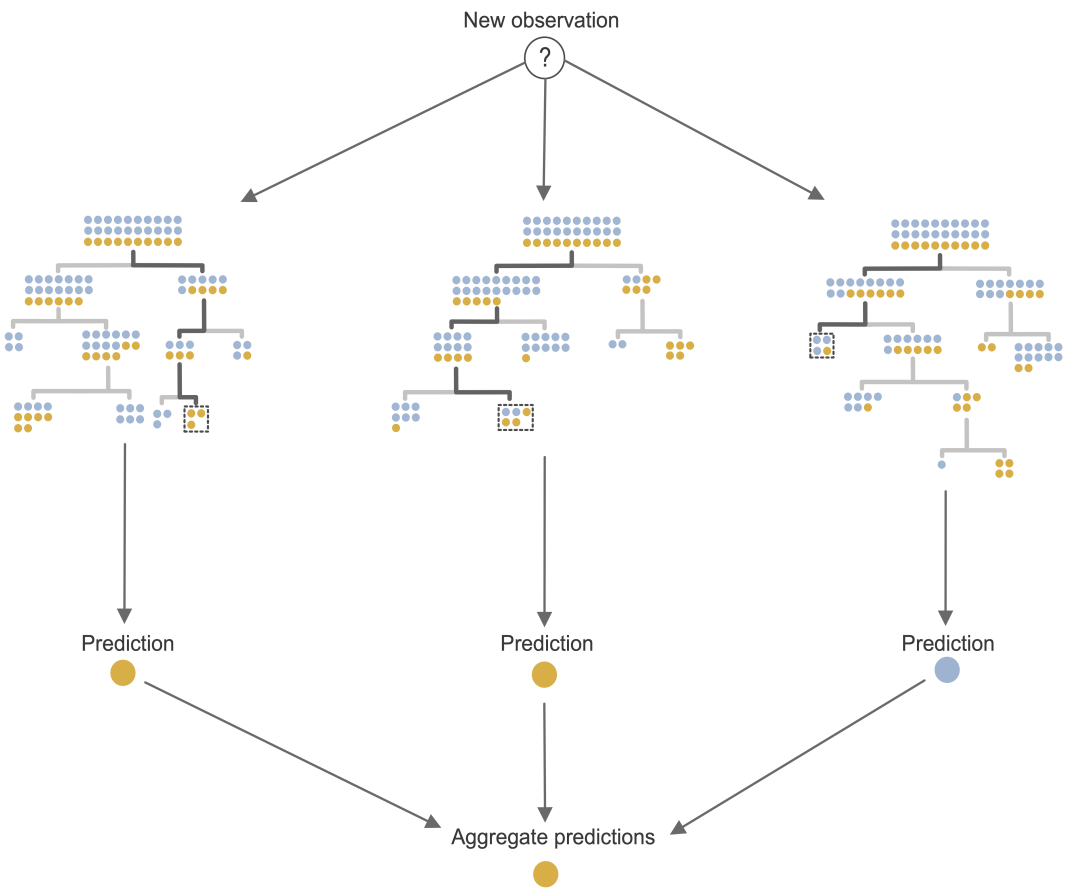
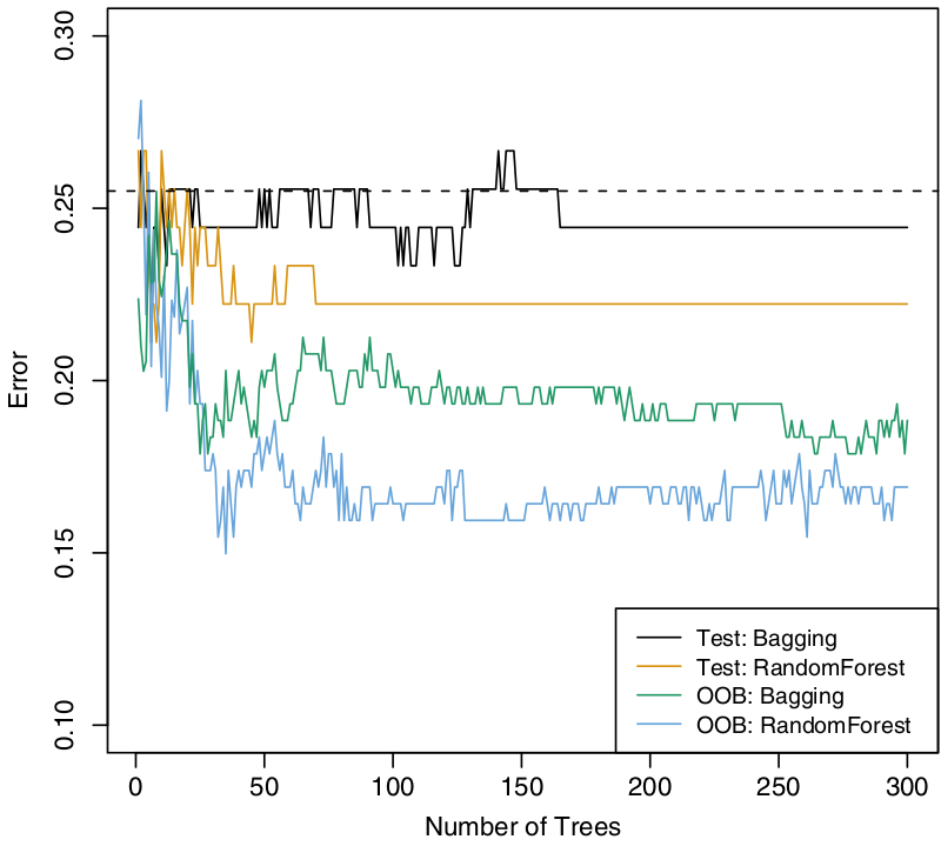


Figure 12: An RF fit based on an ensemble of just three trees is depicted. Source: https://vdsbook.com/12-rf

Bagging vs. Random Forests

Bagging: Bootstrapping for creating various subsets of the training data and averaging predictions from trees built on these subsets. Each tree is built using all features at each split.

Random Forests = Bagging + Random Feature Selection at Splits



Random Forests: Key Hyperparameters

Main tuning parameters:

Hyperparameter	Role	R (`ranger`)	Python (`sklearn`)
Number of variables tried at each split	Controls randomness and tree diversity	`mtry` default: $\lfloor \sqrt{p} \rfloor$	`max_features` default: $\sqrt{p}$
Maximum depth	Limits number of sequential splits; smaller depth regularizes	`max.depth` default: unlimited	`max_depth` default: unlimited
Minimum node size / minimum samples to split	Stops further splitting in small nodes; larger values regularize	`min.node.size` defaults depend on task	`min_samples_split` default: 2
Number of trees	More trees reduce variance but increase computation time	`num.trees` default: 500	`n_estimators` default: 100

Practical takeaway:

- Tune these hyperparameters using cross-validation
- In practice, default values often work reasonably well
- Start with defaults, then tune only if predictive performance needs improvement

Code Example 4: Random Forests in Gretl

File: `../code/04_random_forest.inp`

```
#
    ==================================================================
# Example 4: Random Forests with Feature Importance
# Note: Uses R integration (randomForest package)
# Author: PhD Seminar, Ancona 2026
#
    ==================================================================

# Load data
open ../data/TODO_dataset.gdt

# Prepare data matrices
# TODO: Create X (features) and y (target) matrices

# Use R integration to fit Random Forest
```

Feature Importance

- Individual trees are easy to interpret.
- But how do we evaluate the relevance of high-dimensional (multiple features) ensembles (multiple models)?
- The importance of a variable is measured by the average reduction in error achieved by using this variable for splits in the trees.
- Various importance measures exist, e.g.
  1. permutation importance score
  2. Gini importance score
  3. Impurity-based feature importance (tends to overestimate the importance)
- Other measures include SHAP values, LIME, and partial dependence plots.
- Relevant topic, generally known under the term *Explainable AI*.

Permutation Feature Importance Algorithm

1. **Inputs:** Fitted predictive model, tabular dataset
2. **Compute baseline performance:** Calculate reference score S on unmodified data (e.g., accuracy, MSE etc.)
3. **For each feature X_j:**
   1. For $r = 1$ to R repetitions:
      1. Create corrupted dataset by randomly shuffling rows of only feature X_j
      2. Compute performance score S_r^j on this corrupted data
   2. Calculate importance I_j for feature X_j as the decrease in performance:

$$I_j = S - \underbrace{\frac{1}{R}\sum_{r=1}^{R} S_r^j}_{\text{avg. score of permuted data}} \tag{4}$$

4. **Result:** Higher importance value indicates more influential features

Permutation Feature Importance Visualization

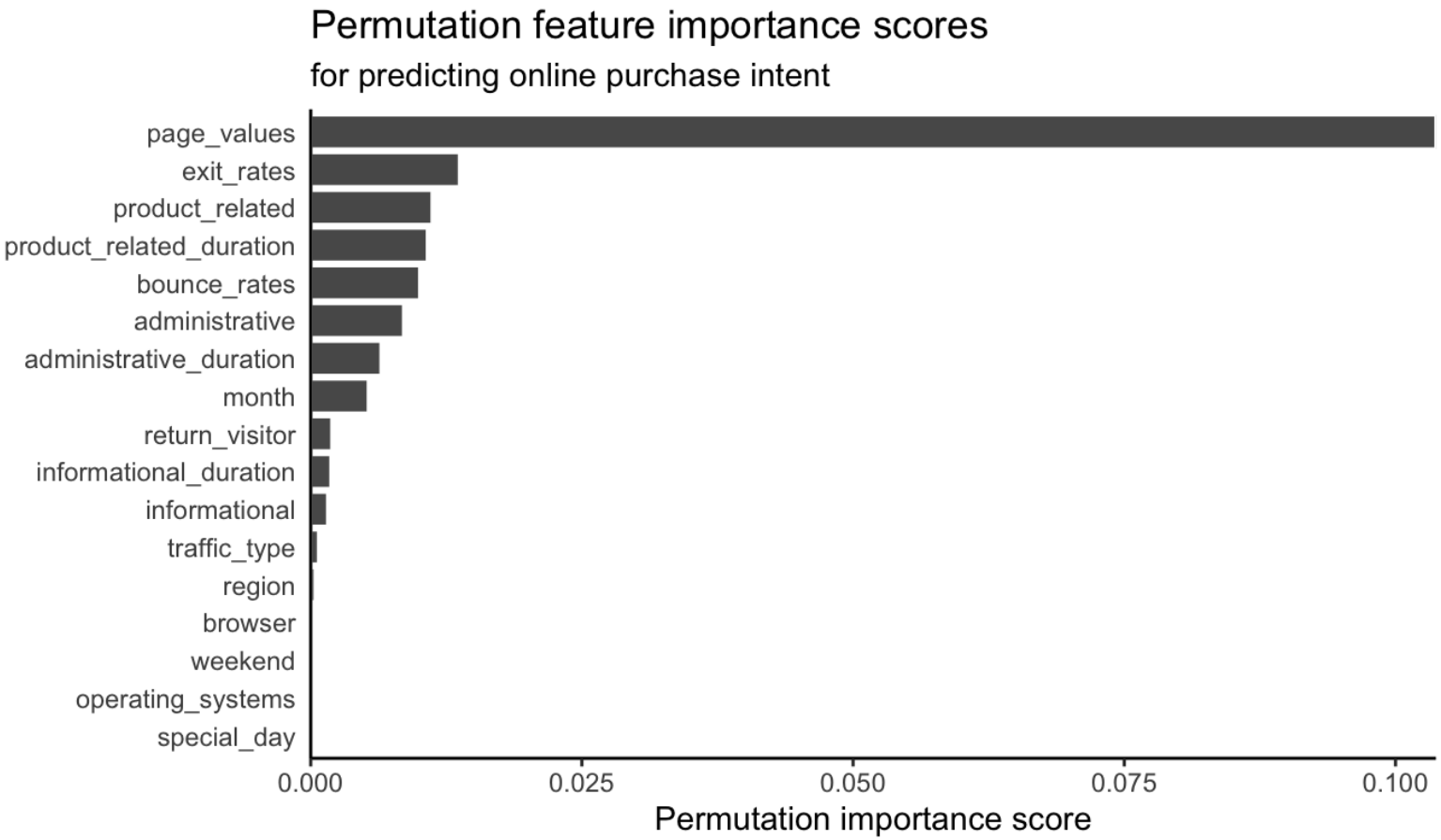


Figure 14: A bar chart showing the permutation feature importance of each variable in the RF fit for predicting online purchase intent. Source: https://vdsbook.com/12-rf#fig-shopping-variable-importance-permutation

Boosting

Question: Can one combine multiple *weak learners* in such a way that the ensemble becomes a *strong learner*?[2]

- Recall, bagging involves training each tree on a bootstrap data set, independent of the other trees.
- With boosting, each model is grown sequentially **using information from previously trained models**.
- Boosting combines multiple weak learners to produce a form of committee whose performance can be significantly better than that of any of the base classifiers $\Rightarrow$ Bias reduction
- Adaptive boosting (AdaBoost) is a strategy developed by Freund and Schapire (1996) for classification.
- Friedman (2001) introduced gradient-based boosting.

[2] A weak learner is a simple model (in terms of complexity measured by the number of features, the degree of non-linearity or number of nodes in case of a tree) whose accuracy is just a little bit better than random guess.

Gretl Package for Random Forests

Package: random_forest.gfn
Author: Allin Cottrell
Version: 0.3 (December 2023)
Installation: `pkg install random_forest`

Main Features:

- Wraps R's `randomForest` package (requires R $\geq$ 4.1.0)
- Supports both regression and classification tasks
- Main function: `random_forest(y, X, opts)`
- Feature importance scoring and visualization
- Flexible train/test split via *n_train* or dummy series
- Automatic hyperparameter tuning (mtry) via `tuneRF`
- GUI interface available

Key Functions:

- `random_forest()`: Train and predict with random forests
- `rf_print()`: Display model results and feature importance
- `rf_plot()`: Visualize feature importance (horizontal bar chart)

Technische Hochschule
Brandenburg
University of
Applied Sciences
Fachbereich
Wirtschaft

Support Vector Machines (SVM)

Support Vector Machines (SVM): Core Idea

Goal: Find a separating hyperplane that maximizes the margin between classes.

Linear SVM (for separable data):

$$\min_{\beta,b} \frac{1}{2}\|\beta\|^2 \quad \text{subject to} \quad y_i(\mathbf{x}_i^\top \beta + b) \geq 1 \text{ for all } i \tag{5}$$

- **Margin**: Distance from hyperplane to nearest points (support vectors)
- Maximize margin $\Leftrightarrow$ minimize $\|\beta\|^2$
- Support vectors are critical observations; ignore far points

Non-separable case (soft margin):

$$\min_{\beta,b,\xi} \frac{1}{2}\|\beta\|^2 + C\sum_{i=1}^{n} \xi_i \tag{6}$$

- Allow misclassifications (slack variables ξ_i)
- Cost parameter C trades off margin width vs. misclassification tolerance

Technische Hochschule
Brandenburg
University of
Applied Sciences
Fachbereich
Wirtschaft

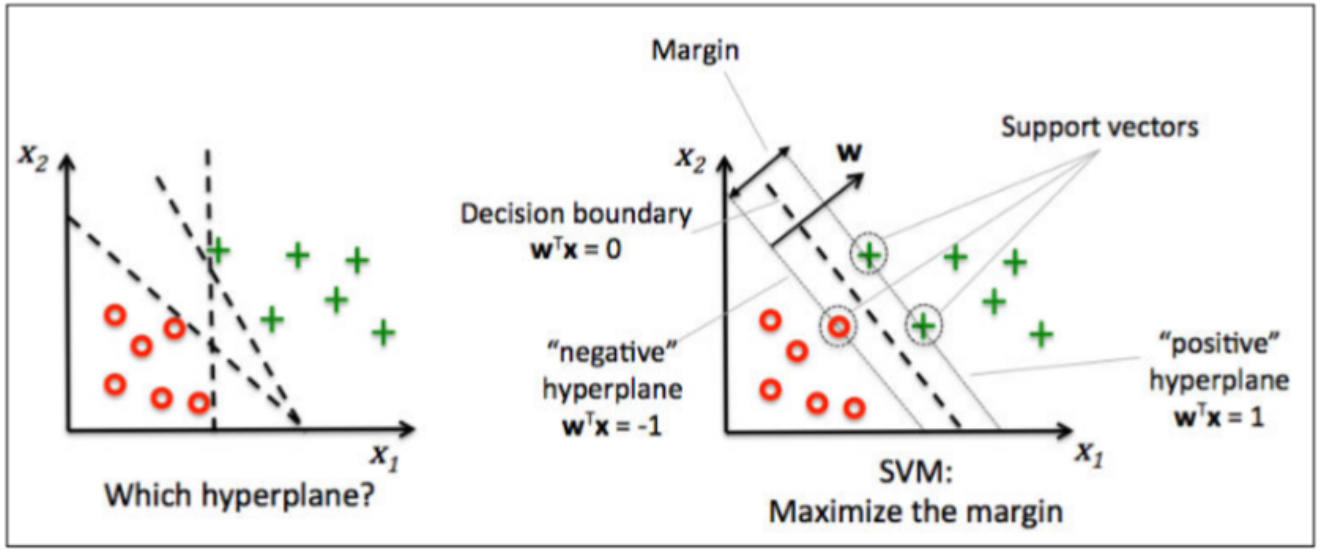


Figure 15: SVM margin and support vectors. Source: (Raschka and Mirjalili, 2019, p. 69).

An open question in the field: Multiclass SVMs — how to extend binary SVM to multi-class problems? Common approaches include one-vs-rest and one-vs-one strategies. For details, see Bishop (2016, p. 338).

Support Vector Machines: Nonlinear Kernels

Linear kernels work only for linearly separable data

Kernel Trick: Transform data to high-dimensional space implicitly (Raschka and Mirjalili, 2019, pp. 75)

Common kernels:

Kernel	**Formula**	**Use Case**
Linear	$K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$	Linearly separable
Polynomial	$K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^\top \mathbf{x}_j + r)^d$	Moderate nonlinearity
RBF (Gaussian)	$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \lVert \mathbf{x}_i - \mathbf{x}_j \rVert^2)$	Highly nonlinear
Sigmoid	$K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\kappa \mathbf{x}_i^\top \mathbf{x}_j + \theta)$	Neural-inspired

Hyperparameter tuning: Kernel choice and parameters (e.g., γ, d) crucial — select via cross-validation

SVM for Regression

Support Vector Regression (SVR): Similar to classification but predicts continuous values

$$\min_{\beta, b, \xi} \frac{1}{2}\|\beta\|^2 + C\sum_{i=1}^{n}(\xi_i + \xi_i^*)$$

subject to: $|y_i - (\mathbf{x}_i^\top \beta + b)| \leq \varepsilon + \xi_i$ and similar for ξ_i^*

- **Epsilon (ε)**: Insensitive zone — errors $< \varepsilon$ are ignored (Hinge loss)
- **Sparse predictions**: Only observations outside ε band are support vectors
- Particularly useful for high-dimensional regression

Gretl Built-in SVM Functionality

Author: Allin Cottrell
Library: libsvm (integrated plugin, no external installation needed)

Main Features:

- Built-in function: `svm(list L, bundle params)`
- Supports both regression and classification
- Flexible train/test split via *n_train* parameter
- Comprehensive cross-validation for hyperparameter tuning (grid/line search)
- MPI support for large-scale parallel cross-validation

Key SVM Types & Kernels:

- Classification: C-SVC (cost-based) or ν-SVC (fraction of support vectors)
- Regression: ϵ-SVR (margin-based) or ν-SVR (support vector fraction)
- Kernels: linear, polynomial (with degree parameter), RBF, sigmoid

Code Example 5: Support Vector Machines in Gretl

File: `../code/05_svm.inp`

```
# ==================================================================
# Example 5: Support Vector Machines (libsvm integration)
# Dataset: TODO - ISL classification/regression dataset
# Author: PhD Seminar, Ancona 2026
# ==================================================================


# Load data
open ../data/TODO_dataset.gdt

# TODO: Prepare training and test data
# Standardize features (important for SVM)
```

Tree-Based vs. Kernel Methods: Comparison

Aspect	Trees / Random Forests	Kernel Methods (SVM, KNN)
Interpretability	High	Low
Feature importance	Built-in (Gini, permutation)	Difficult to extract
Computational cost	Moderate	High (especially SVM)
Nonlinearity	Automatic (recursive splits)	Via kernel choice
Scaling features	Not necessary	Essential
High dimensions	Possible	Better (with RBF kernel)
Interaction effects	Captures naturally	Needs manual specification

Rule of thumb:

- **Trees/RF**: Start here for interpretability, feature importance
- **SVM/KNN**: Try when trees underperform or data is very high-dimensional

Comparison of Machine Learning Methods

Characteristic	SVM	CART	GAM	KNN	Gradient Boost
Natural handling of data of "mixed" type	poor	✓✓	✓✓	poor	✓✓
Handling of missing values	poor	✓✓	poor	✓✓	✓✓
Robustness to outliers in input space	poor	✓✓	✓	✓✓	✓✓
Insensitive to monotone transformations of inputs	poor	✓✓	✓✓	poor	✓✓
Computational scalability (large N)	poor	✓✓	✓✓	poor	✓✓
Ability to deal with irrelevant inputs	poor	✓✓	✓	poor	✓✓
Ability to extract linear combinations of features	✓✓	poor	poor	✓	✓
Interpretability	poor	✓	✓✓	poor	✓✓
Predictive power	✓✓	poor	poor	✓✓	✓✓

Table 1: Comparison of characteristics across different machine learning methods.

Technische Hochschule
Brandenburg
University of
Applied Sciences
Fachbereich
Wirtschaft

Cross-Validation and Model Evaluation

Robust Performance Assessment

Training and Test Errors

We distinguish between:

1. **Training error**: Average error between realization and prediction on the training data (*In-Sample Error*).
2. **Test (Generalization) error**: Average error when predicting unseen test data (also *Out-of-Sample* prediction).

Goal: The test error should be as small as possible.

The training error can dramatically underestimate the test error.

For model selection and evaluation, estimate the test error.

Why Cross-Validation?

- **Single test set problem**: With one split, results are unstable (depends on lucky/unlucky split)
- **Data scarcity**: May not have enough data to dedicate large portion to testing
- **Better generalization estimate**: Average error across multiple folds more reliable
- **Hyperparameter tuning**: Need unbiased estimate of performance for parameter selection
- **Model agnostic**: Works for any model

Core principle: Avoid using the same data for training and evaluation

k-Fold Cross-Validation I

- Split the available data into k groups.
- $k-1$ of the groups are used to train a model.
- Evaluate the model on the remaining validation group using a <METRIC>.
- Repeat the steps for all k (typically 5 or 10) possible choices.
- The calculated evaluation metrics are averaged to estimate the error rate.
- LOOCV is a special case of k-fold CV, where $k = n$.

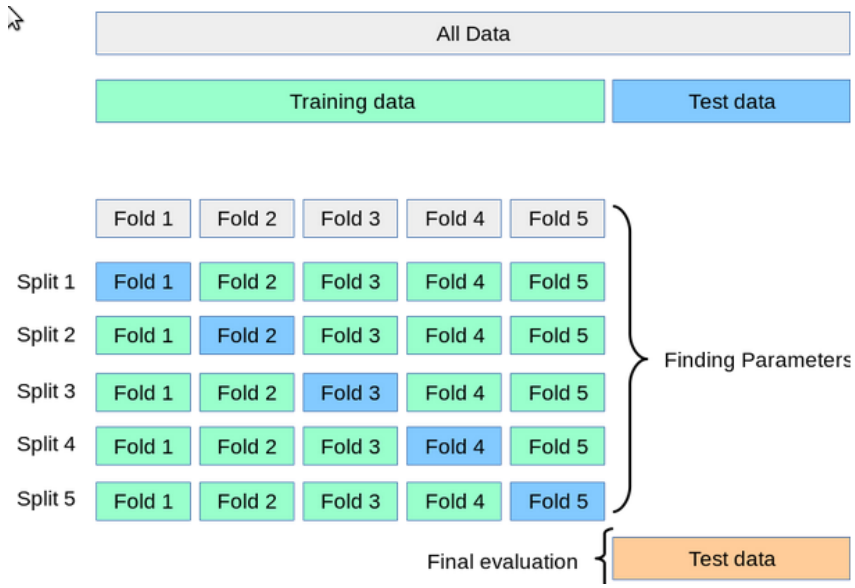


Figure 16: Illustration of k-fold CV for $k = 5$. The training data is shown in green. The held-out set is indicated by the blue block.

Comparison of LOOCV and k-Fold CV

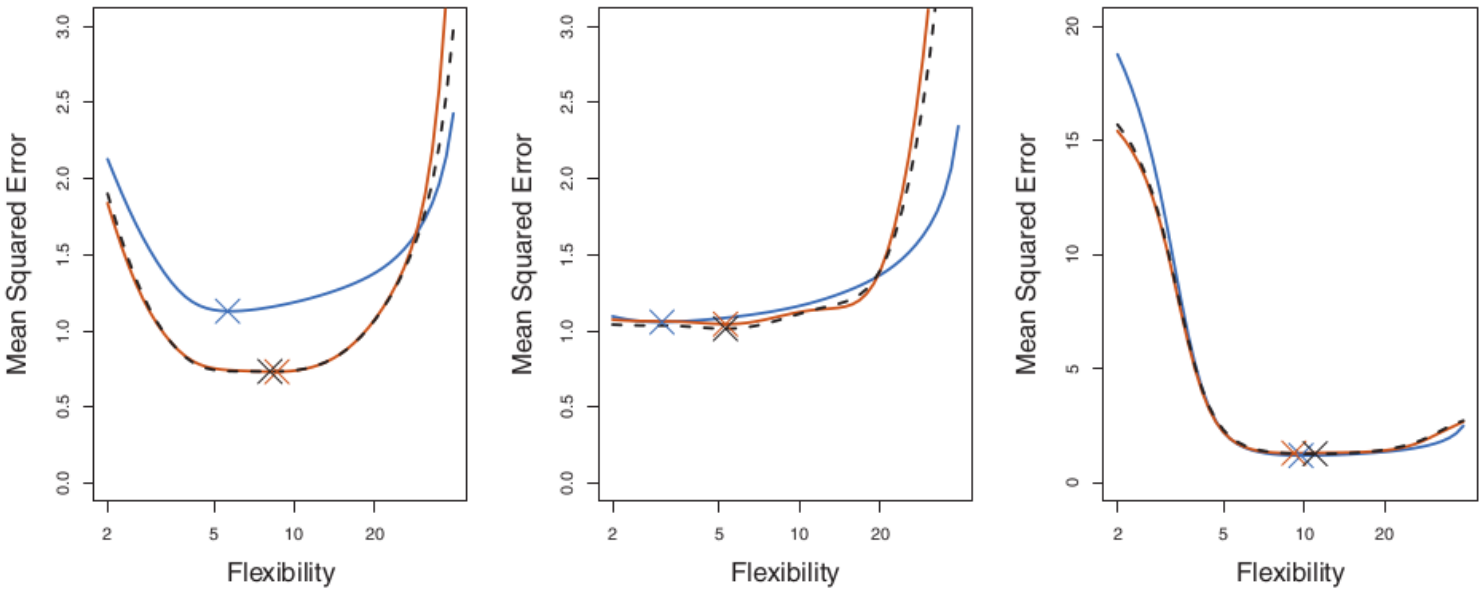


FIGURE 5.6. *True and estimated test MSE for the simulated data sets in Figures 2.9* (left)*, 2.10* (center)*, and 2.11* (right)*. The true test MSE is shown in blue, the LOOCV estimate is shown as a black dashed line, and the* 10*-fold CV estimate is shown in orange. The crosses indicate the minimum of each of the MSE curves.*

Figure 17: Similar test error rates achieved by LOOCV and k-fold CV. Even though the test error may be underestimated, the correct hyperparameter value that minimizes the MSE can be well estimated by CV. Source: James et al. (2023, p. 208).

Classification Problems

- CV is also useful for classification problems.
- The previously described CV techniques are analogous to what was described before.
- However, the loss function needs to change.
- For classification, we could calculate the classification error as follows:

$$e_i = \mathbf{1}(y_i \neq \hat{y}_i) \tag{7}$$

  which is known as "zero-one loss."
- The LOOCV test error rate then becomes:

$$CV_n = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}(y_i \neq \hat{y}_i) \tag{8}$$

Cross-Validation

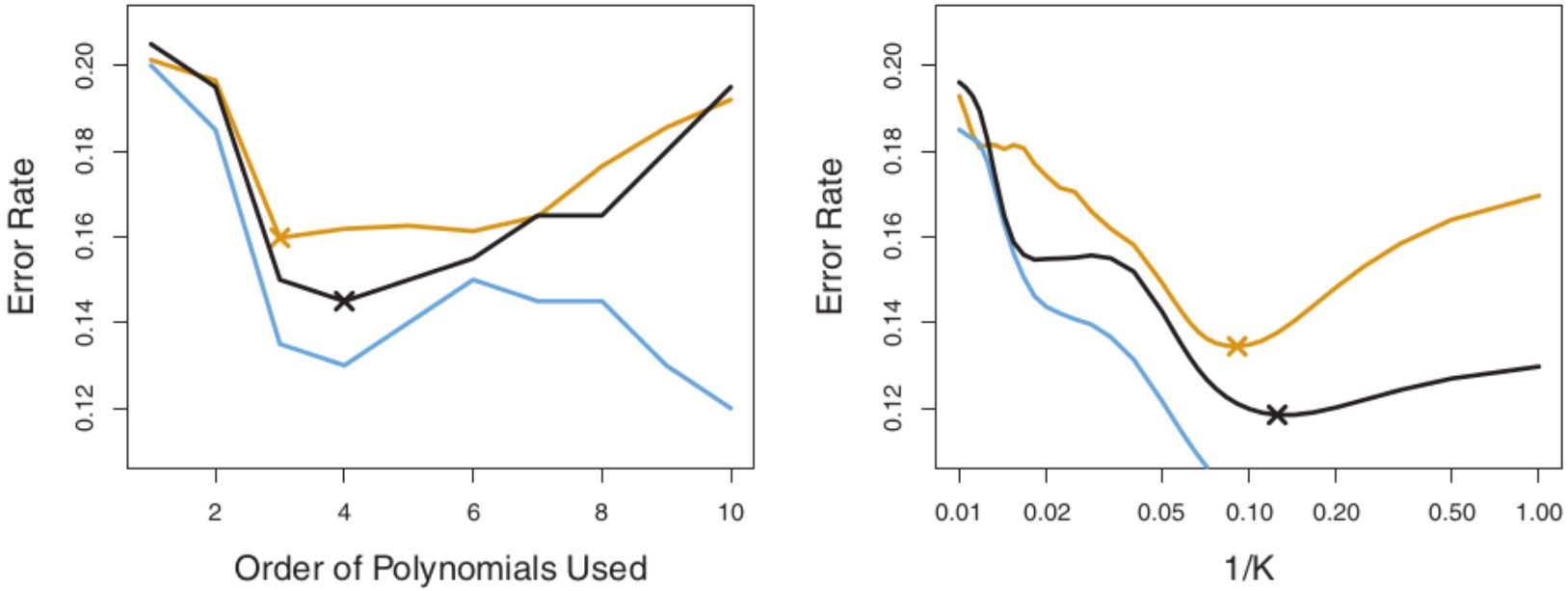


FIGURE 5.8. *Test error (brown), training error (blue), and 10-fold CV error (black) on the two-dimensional classification data displayed in Figure 5.7.* Left: *Logistic regression using polynomial functions of the predictors. The order of the polynomials used is displayed on the x-axis.* Right: *The KNN classifier with different values of K, the number of neighbors used in the KNN classifier.*

Source: James et al. (2023, p. 211)

Imbalanced Class Distribution

- Some classification problems may have a large imbalance in the distribution of target classes.
- For example, there could be many times more negative than positive samples.
- In such cases, stratified sampling is used to ensure that the relative class frequencies are approximately preserved in each train and validation fold.

$\Rightarrow$ Use stratified k-fold CV.

Imbalanced Class Distribution

3.1.2.2.1. Stratified k-fold

`StratifiedKFold` is a variation of *k-fold* which returns *stratified* folds: each set contains approximately the same percentage of samples of each target class as the complete set.

Here is an example of stratified 3-fold cross-validation on a dataset with 50 samples from two unbalanced classes. We show the number of samples in each class and compare with `KFold`.

```
>>> from sklearn.model_selection import StratifiedKFold, KFold
>>> import numpy as np
>>> X, y = np.ones((50, 1)), np.hstack(([0] * 45, [1] * 5))
>>> skf = StratifiedKFold(n_splits=3)
>>> for train, test in skf.split(X, y):
...     print('train - {}   |   test - {}'.format(
...         np.bincount(y[train]), np.bincount(y[test])))
train - [30  3]   |   test - [15  2]
train - [30  3]   |   test - [15  2]
train - [30  4]   |   test - [15  1]
>>> kf = KFold(n_splits=3)
>>> for train, test in kf.split(X, y):
...     print('train - {}   |   test - {}'.format(
...         np.bincount(y[train]), np.bincount(y[test])))
train - [28  5]   |   test - [17]
train - [28  5]   |   test - [17]
train - [34]   |   test - [11  5]
```

We can see that `StratifiedKFold` preserves the class ratios (approximately 1 / 10) in both train and test dataset.

Source: https://scikit-learn.org/stable/modules/cross_validation.html#stratified-k-fold

Cross-Validation of Time Series Data (and Panel Data)

- Time series exhibit inherent serial correlation and are potentially non-stationary.
- Applying CV is not straightforward and is typically replaced by out-of-sample evaluation.
- The training set consists only of observations that occur before the period to be forecasted.
- Therefore, no future observations can be used when creating the forecast.

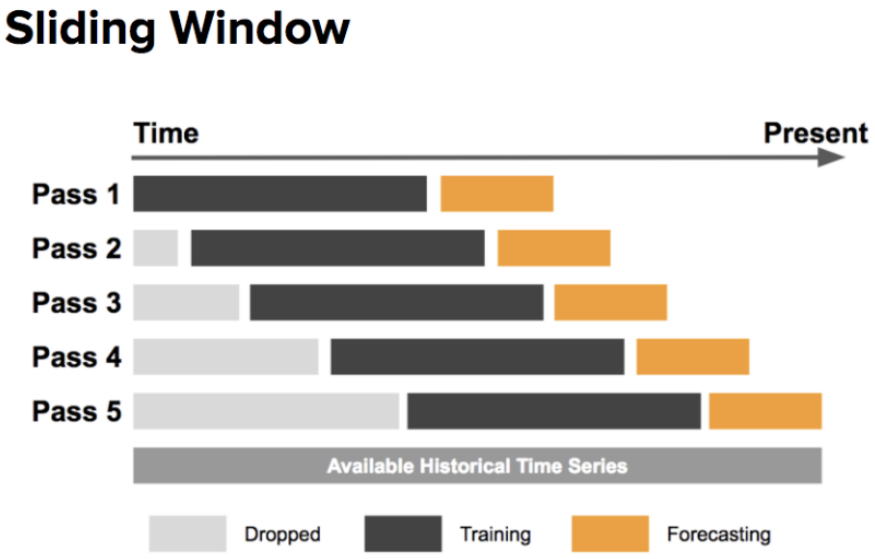


Figure 18: Cross-validation procedure with a 'rolling' window for training. The diagram shows a sequence of h-step ahead forecasts can be created.

Code Example 6: K-Fold Cross-Validation Setup

File: `../code/06_kfold_cv.inp`

```
#
    ==========================================================

# Example 6: K-Fold Cross-Validation Setup
# Package: CvDataSplitter.gfn (Artur Tarassow)
# Author: PhD Seminar, Ancona 2026
#
    ==========================================================



# Install and include package
pkg install CvDataSplitter --quiet
include CvDataSplitter.gfn

# Load data
open ../data/TODO_dataset.gdt
```

Forecast Errors for Regression

- Forecast error as the difference between the observed value y_i and the forecast $\hat{y}_i$:

$$e_i = y_i - \hat{y}_i \tag{9}$$

- Decomposition of the error into different components: Forecast error = Signal + Noise
- Forecast error does not mean 'error' in the sense of 'wrong'.
- Forecast errors represent the non-predictable part of the observations – at least when the model is correctly specified.
- Measurement of forecast performance by summarizing forecast errors in different ways.

Evaluation Metrics for Regression

An overview of some metrics for measuring prediction accuracy:

Metric	**Formula**	**Interpretation**
ME	$\frac{1}{n}\sum(y_i - \hat{y}_i)$	Average error (bias)
RMSE	$\sqrt{\frac{1}{n}\sum(y_i - \hat{y}_i)^2}$	Standard deviation of errors
MAE	$\frac{1}{n}\sum\lvert y_i - \hat{y}_i\rvert$	Average absolute error
MAPE	$\frac{100\%}{n}\sum\left\lvert\frac{y_i - \hat{y}_i}{y_i}\right\rvert$	Average percentage error (scale-free)

For discussions on the advantages and disadvantages of each metric, see Hyndman and Koehler (2006) and Hyndman (2014).

Metrics for Classification Problems

- There are many different metrics for evaluating classification performance.
- The choice depends, among other things, on whether it's a binary or multi-class classification problem.
- Libraries:
  - Python: `sklearn.metrics` module
  - Gretl: `FEP` package
- Measures assume that the predictions are either:
  - binary (0/1) or multi-class (e.g., 0, 1, 2, ...), or
  - probabilities (e.g., $P(y = 1|x)$).

Binary Classification: Evaluation Metrics

	Predicted		
Observed	0	1	**Total**
0	TN	FP	N^*
1	FN	TP	P^*
Total	N^*	P^*	$N + P$

Metric	**Definition**
Accuracy	$\frac{\text{TP}+\text{TN}}{\text{TP}+\text{TN}+\text{FP}+\text{FN}}$ = fraction correct
Precision	$\frac{\text{TP}}{\text{TP}+\text{FP}}$ = of predicted positives, how many correct?
Recall (Sensitivity)	$\frac{\text{TP}}{\text{TP}+\text{FN}}$ = of actual positives, how many detected?
Specificity	$\frac{\text{TN}}{\text{TN}+\text{FP}}$ = of actual negatives, how many correctly rejected?
F1-Score	$2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision}+\text{Recall}}$ = harmonic mean

Where:

- TP = True Positives, TN = True Negatives
- FP = False Positives, FN = False Negatives

Trade-off: Precision vs. Recall depends on context

- High recall: Medical diagnosis (minimize missed cases)
- High precision: Spam filtering (avoid false alarms)

From Binary to Multi-class and Multi-label Classification

- Some metrics are primarily for binary classification tasks (e.g., F1-Score, ROC AUC).
- In multi-class or multi-label problems, the data is treated as a collection of binary problems.
- There are different methods to average binary metrics across multiple classes:
  - **macro**: Calculates the average of binary metrics with equal weight for each class. Useful when rare classes are important.
  - **weighted**: Accounts for class imbalance by weighting each class according to its frequency in the true data.
  - **micro**: Gives each sample-class pair equal contribution to the overall metric. Sums the numerators and denominators of class metrics.
  - **samples**: Only for multi-label problems. Calculates the metric over the true and predicted classes for each sample.
- See Raschka and Mirjalili (2019, pp. 197) and https://scikit-learn.org/stable/modules/model_evaluation.html#from-binary-to-multiclass-and-multilabel

Technische Hochschule
Brandenburg
University of
Applied Sciences
Fachbereich
Wirtschaft

Unsupervised Learning

K-Means Clustering and Determining Optimal k

K-Means Clustering: Algorithm

Goal: Partition n observations into k clusters based on similarity

K-Means Algorithm:

1. Randomly initialize k cluster centroids $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_k$
2. Repeat until convergence:
   - **Assignment step**: Assign each observation to closest centroid
   $$C_k = \{i : \|\mathbf{x}_i - \boldsymbol{\mu}_k\| \leq \|\mathbf{x}_i - \boldsymbol{\mu}_{k'}\| \text{ for all } k' \neq k\}$$
   - **Update step**: Recompute centroids as mean of assigned points
   $$\boldsymbol{\mu}_k \leftarrow \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i$$
3. Converged when assignments no longer change

Minimizes: Within-cluster sum of squares (WCSS)

$$W = \sum_{k=1}^{K} \sum_{i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

Illustration of the iterations

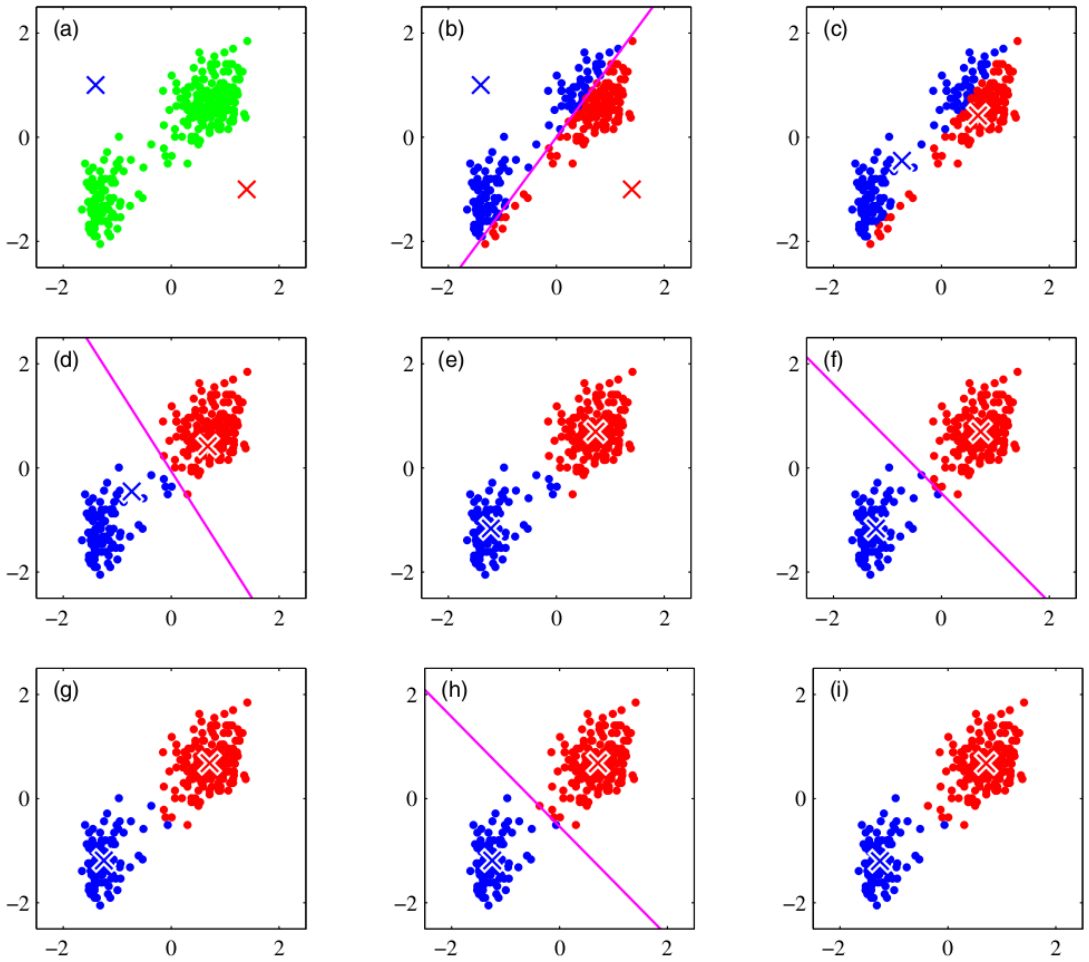


Figure 19: Illustration of the K-means algorithm for $K = 2$ clusters. The centroids are shown as crosses. The algorithm quickly converges to a stable solution. Source: Bishop (2016).

K-Means: Distance Metric and Initialization

Distance Metric:

- Default: Euclidean distance $\|\mathbf{x}_i - \boldsymbol{\mu}_k\| = \sqrt{\sum_j (x_{ij} - \mu_{kj})^2}$
- Assumes continuous features on similar scales
- **Important:** Standardize features before clustering!

Initialization Problem:

- K-Means converges to **local optimum**, not global
- Different random starting points $\Rightarrow$ different final clusters
- **Solution**: Run k-means multiple times with different initializations, keep best
- Alternative: K-Means++ initialization (smarter initial centroids)

How many clusters?

- As clusters are **latent**.
- Widely used are *ad hoc* heuristics such as the 'elbow method'.[3]

Intuition:

- WCSS always decreases as k increases (more clusters fit data better)
- But improvement becomes marginal beyond optimal k

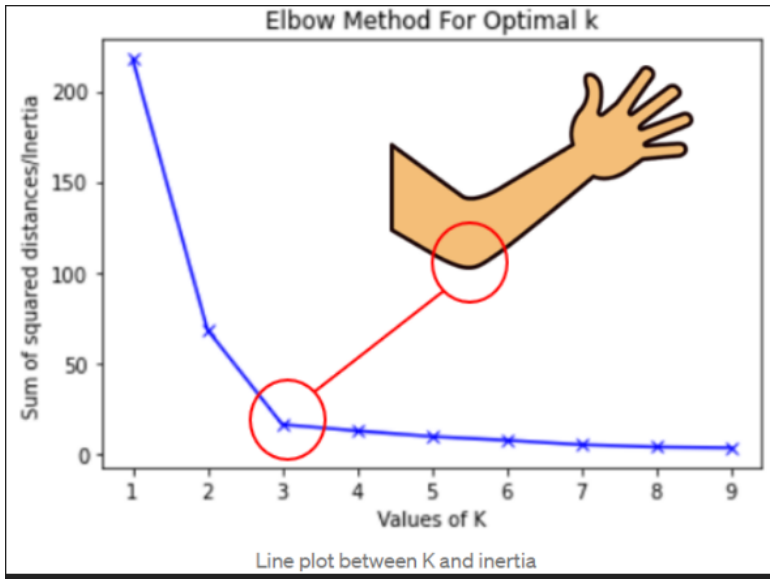


Figure 20: Elbow = point where marginal improvement drops off.

[3] For critique on the Elbow method, see e.g. Schubert (2023) and James et al. (2023, Ch. 12.4).

Silhouette Plot: A Quantitative Approach

Silhouette Coefficient: Measures how similar an observation is to its own cluster vs. other clusters

For observation i:

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)} \tag{10}$$

where:

- a_i = average distance from i to points in same cluster
- b_i = average distance from i to points in nearest other cluster

Interpretation:

- s_i ranges from -1 to $+1$
- $s_i \approx +1$: Well-clustered (far from other clusters)
- $s_i \approx 0$: On boundary between clusters
- $s_i \approx -1$: Possibly misclassified

Use: Silhouette plot shows s_i for all observations; choose k that maximizes average silhouette width.

The 'kmeans' Gretl package

Author: Artur Tarassow
Installation: `pkg install kmeans`

Main Features:

- Implements the k-means clustering algorithm.
- Provides functions to perform k-means clustering on datasets, allowing users to specify the number of clusters and other parameters.
- The package includes functions
  - for visualizing the clustering results,
  - various distance metrics, and
  - evaluating the quality of the clusters formed by means of Silhouette scores.
- Documentation: https://gretl.sourceforge.net/current_fnfiles/kmeans.gfn

Questions?

Thank you!

Questions and Discussion

Bibliography I

- Bishop, C. M. (2016). *Pattern Recognition and Machine Learning*. New York, NY: Springer New York.
- Breiman, L. (1996). "Bagging Predictors". In: *Machine Learning* 24.2, pp. 123–140. DOI: 10.1007/BF00058655.
- — (2001). "Statistical Modeling: The Two Cultures". In: *Statistical Science* 16.3, pp. 199–231.
- Breiman, L. et al. (1984). *Classification and RegressionTrees.* Hall/CRC: Chapman.
- Freund, Y. and R. E. Schapire (1996). "Experiments with a New Boosting Algorithm". In: *Thirteenth International Conference on Machine Learning*. Ed. by L. Saitta. Morgan Kaufmann.
- Friedman, J. H. (2001). "Greedy Function Approximation: A Gradient Boosting Machine.". In: *Annals of Statistics* 29.5, pp. 1189–1232.

Bibliography II

- Hastie, T., R. Tibshirani, and J. Friedman (2009). *The Elements of Statistical Learning. Data Mining, Inference, and Prediction*. 2nd ed. Springer.
- Hyndman, R. J. (2014). "Measuring Forecast Accuracy". In.
- Hyndman, R. J. and A. B. Koehler (2006). "Another Look at Measures of Forecast Accuracy". In: *International Journal of Forecasting* 22.4, pp. 679–688. DOI: 10.1016/j.ijforecast.2006.03.001.
- James, G. et al. (2023). *An Introduction to Statistical Learning: With Applications in Python*. 1st ed. 2023 edition. Cham, Switzerland: Springer.
- Raschka, S. and V. Mirjalili (2019). *Python Machine Learning: Machine Learning and Deep Learning with Python, scikit-learn, and TensorFlow 2*. 3rd ed. Birmingham Mumbai: Packt Publishing.

Bibliography III

- Schubert, E. (2023). "Stop Using the Elbow Criterion for K-Means and How to Choose the Number of Clusters Instead". In: *ACM SIGKDD Explorations Newsletter* 25.1, pp. 36–42. DOI: 10.1145/3606274.3606278.
- Yu, B. and R. L. Barter (2024). *Veridical Data Science: The Practice of Responsible Data Analysis and Decision Making*. Cambridge, Massachusetts: The MIT Press.
- Zou, H. (2006). "The Adaptive Lasso and Its Oracle Properties". In: *Journal of the American Statistical Association* 101.476, pp. 1418–1429. DOI: 10.1198/016214506000000735.

Loss Functions and Objective

Different loss functions for different goals:

Method	**Loss Function**	**Interpretation**
OLS	$\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$	Squared error
Robust Regression	$\sum_{i=1}^{n}\lvert y_i - \hat{y}_i\rvert$	Absolute error (less sensitive to outliers)
Logistic Regression	$-\sum[y_i \log(\hat{p}_i) + (1 - y_i)\log(1 - \hat{p}_i)]$	Cross-entropy (classification)
Hinge Loss (SVM)	$\sum \max(0, 1 - y_i \hat{f}(x_i))$	Margin-based (robust)

ROC Curve for Binary Classification I

- ROC stands for 'Receiver Operating Characteristics' and originates from communication theory.
- The ROC curve is a common graphical representation to display the two types of errors for all possible thresholds c.
- The overall performance of a classifier is given by the area under the curve (AUC).
- The statistics can be applied independently of the classifier used.
- The AUC is 0.5 for a classifier that performs no better than random chance.

ROC Curve for Binary Classification II

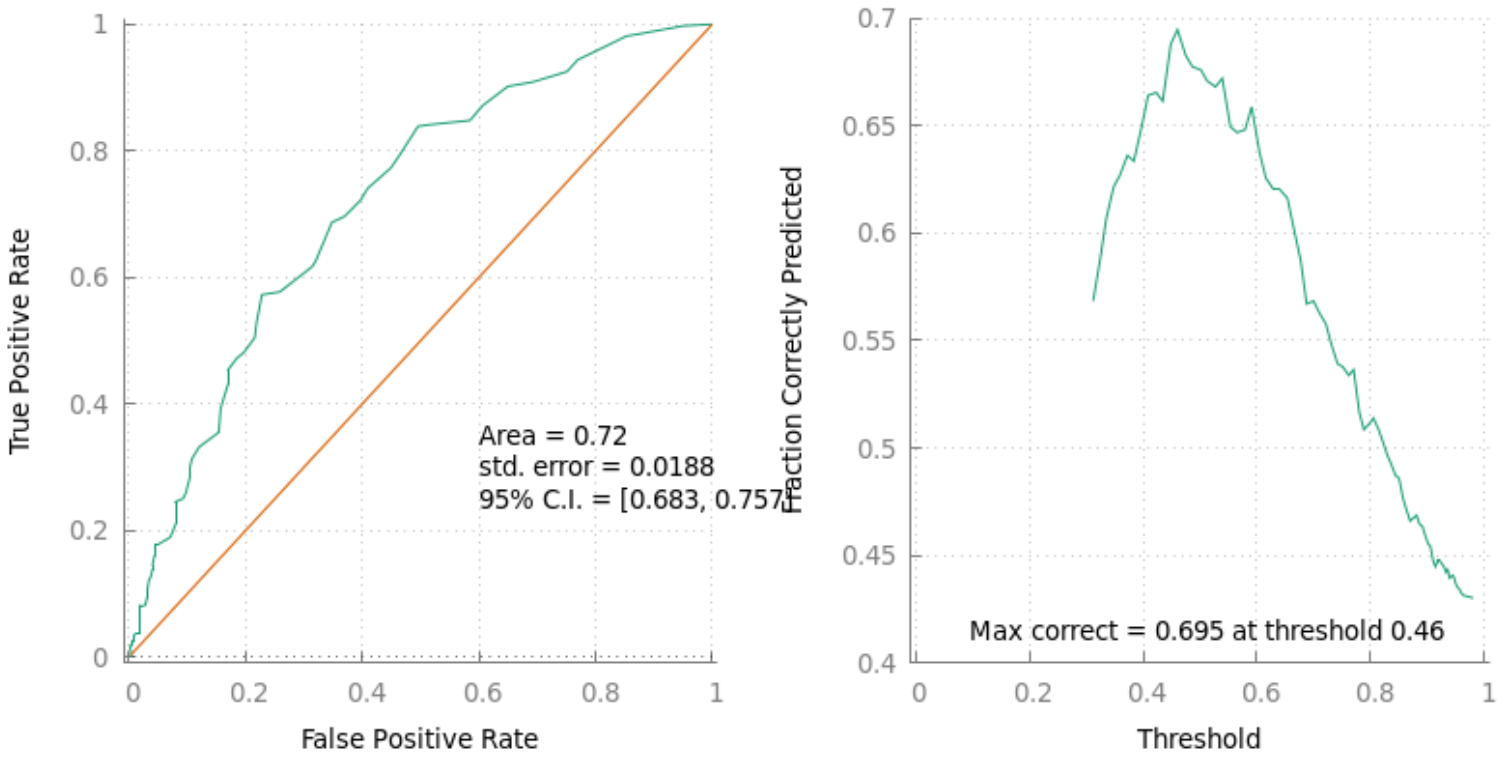


Figure 21: A ROC curve for a Logit model with the `mroz87` data. **Left**: The curve traces two types of errors as the threshold c varies. Each combination shows the true positive rate and false positive rate as a function of c. The ideal ROC curve hugs the upper left corner, showing a high true positive rate and a low false positive rate. The 45-degree line represents the 'no information' classifier (pure guessing). **Right**: This graph shows the proportion of correctly predicted classes as a function of the threshold parameter c. The maximum is reached at $c = 0.46$, which is only slightly different from the standard threshold of 0.5.

Brier Score Loss for Probabilistic Classification

- The Brier Score (BS) loss function is applicable for binary classes.
- The BS is a "proper score function" that measures the accuracy of probabilistic predictions.
- The function calculates the mean squared error between the actual outcome and the predicted probability estimate.
- The value is between 0 and 1, with a lower value (smaller mean squared difference) indicating a more accurate prediction.

$$\text{Brier Score} = \frac{1}{n}\sum_{i=1}^{n}(y_i - p_i)^2 \tag{11}$$

where $p = \Pr(y = 1)$ and $y \in \{0, 1\}$.

sklearn method: https://scikit-learn.org/stable/modules/model_evaluation.html#brier-score-loss